



**COMSATS University Islamabad
Abbottabad, Pakistan**

AI-Based Cyberbullying Detection App

By

Hafsa Khurram CIIT/FA22-BCS-058/ATD

Zainab Asif CIIT/FA22-BCS-068/ATD

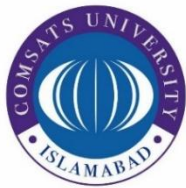
Eiman Tahir CIIT/FA22-BCS-071/ATD

Supervisor

Nauman Khan

Bachelor of Science in Computer Science (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



**COMSATS University Islamabad
Abbottabad, Pakistan**

AI-Based Cyberbullying Detection App

**A project presented to
COMSATS University Islamabad
Abbottabad Campus, Pakistan**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

Hafsa Khurram CIIT/FA22-BCS-058/ATD

Zainab Asif CIIT/FA22-BCS-068/ATD

Eiman Tahir CIIT/FA22-BCS-071/ATD

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Hafsa Khurram

Zainab Asif

Eiman Tahir

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Project title” was developed by **HAFSA KHURRAM (CIIT/FA22-BCS-058)**, **ZAINAB ASIF (CIIT/FA22-BCS-068)** and **EIMAN TAHIR (CIIT/FA22-BCS-071)**, under the supervision of “**NAUMAN KHAN**” that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

In today's digital environment, children frequently use online platforms for communication, learning, and social interaction. However, these platforms expose them to risks such as cyberbullying, harassment, and inappropriate content. Traditional monitoring methods rely on parents or teachers to manually supervise conversations, which leads to several issues including lack of real-time monitoring, human limitations in tracking large volumes of chats, delayed response to harmful situations, and absence of proper evidence for future reference.

Talking about the current solutions, most systems depend on simple keyword filtering or post-incident reporting, regardless of whether harmful content is detected in time or not. Moreover, these systems do not provide intelligent analysis or proactive protection, and they often fail to cover all interactions effectively, requiring constant manual oversight.

To overcome these problems, the KidSafe AI-Based Cyberbullying Detection App is developed. It is an intelligent platform that provides a safe and supervised chat environment for children. The system monitors conversations in real time using Artificial Intelligence and Natural Language Processing techniques to detect toxic, abusive, or threatening language before it reaches the child. Based on severity, messages can be allowed, masked, or blocked, and alerts are generated for parents and teachers.

KidSafe is a desktop and mobile-based application that supports multiple user roles including children, parents, teachers, and administrators. It provides monitoring dashboards for parents and teachers to review flagged messages, receive alerts, and track user behavior. The system also allows reporting, complaint handling, and provides educational resources to promote safe online communication.

It is a modular system consisting of a Blazor .NET MAUI frontend, ASP.NET Core backend services, a Python-based AI detection module, and a SQLite database for secure data storage. Communication between components is handled through APIs and real-time notification services, allowing users to monitor activities and respond effectively.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**NAUMAN KHAN**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Hafsa Khurram

Zainab Asif

Eiman Tahir

ABBREVIATIONS

Abbreviation	Full Form
AIMS	Artificial Intelligence Moderation Service
API	Application Programming Interface
ASP.NET	Active Server Pages .NET
CLR	Common Language Runtime
CMS	Content Moderation System
DOM	Document Object Model
DTO	Data Transfer Object
EF Core	Entity Framework Core
FCM	Firebase Cloud Messaging
Git	Global Information Tracker
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
IoC	Inversion of Control
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model-View-Controller
NLP	Natural Language Processing
ORM	Object-Relational Mapping
REST	Representational State Transfer
REST API	Representational State Transfer Application Programming Interface
RTM	Real-Time Messaging
SDK	Software Development Kit
SPA	Single Page Application
SQLite	Structured Query Language Lite
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format – 8-bit
MAUI	Multi-platform App UI

TABLE OF CONTENTS

<u>1</u>	<u>Introduction</u>	13
1.1	<u>Brief</u>	13
1.2	<u>Relevance to Course Modules</u>	14
1.3	<u>Project Background</u>	14
1.4	<u>Literature Review</u>	15
1.5	<u>Analysis from Literature Review</u>	15
1.6	<u>Methodology and Software Lifecycle for This Project</u>	16
1.6.1	<u>Rationale behind Selected Methodology</u>	16
<u>2</u>	<u>Problem Definition</u>	17
2.1	<u>Problem Statement</u>	17
2.2	<u>Deliverables and Development Requirements</u>	17
2.2.1	<u>Deliverables</u>	17
2.2.2	<u>Development Requirements</u>	18
<u>3</u>	<u>Requirement Analysis</u>	19
3.1	<u>Use Cases Diagram</u>	19
3.1.1	<u>Use Case Description</u>	21
3.2	<u>Detailed Use Case</u>	21
3.3	<u>Functional Requirements</u>	41
3.3.1	<u>User Management Module</u>	41
3.3.2	<u>Chat and Messaging Module</u>	41
3.3.3	<u>Cyberbullying Detection and Warnings Module</u>	41
3.3.4	<u>Parental/Teacher Dashboard</u>	41
3.3.5	<u>Emergency Alert System</u>	42
3.3.6	<u>Educational and Support Module</u>	42
3.3.7	<u>Notifications Module</u>	42
3.3.8	<u>Achievement and Progress Module</u>	42
3.3.9	<u>Classroom and Enrollment Module</u>	42
3.3.10	<u>Moderation Module</u>	42
3.3.11	<u>Feedback and Complaint Module</u>	42
3.3.12	<u>Educational Resources Module</u>	43
3.4	<u>Non-Functional Requirements</u>	43

3.4.1	<u>Performance</u>	43
3.4.2	<u>Security</u>	43
3.4.3	<u>Usability</u>	43
3.4.4	<u>Scalability</u>	43
3.4.5	<u>Reliability</u>	43
3.4.6	<u>Compliance</u>	43
4	<u>Design and Architecture</u>	44
4.1	<u>System Architecture</u>	44
4.1.1	<u>User Interface Layer</u>	44
4.1.2	<u>Application Logic Layer</u>	44
4.1.3	<u>Data Storage Layer</u>	44
4.1.4	<u>Modules and Relationships User Management</u>	44
4.1.5	<u>Chat & Communication</u>	45
4.1.6	<u>AI Safety Filter</u>	45
4.1.7	<u>Alerts & Notifications</u>	45
4.1.8	<u>Report Management</u>	45
4.1.9	<u>Emergency Support</u>	45
4.1.10	<u>Consultation & Support System</u>	45
4.1.11	<u>Document & Media Management</u>	46
4.1.12	<u>Feedback & Complaint System</u>	46
4.2	<u>Data Representation</u>	46
4.2.1	<u>Relationships Overview</u>	47
4.3	<u>Process Flow/Representation</u>	46
4.3.1	<u>Login–Secure User Authentication</u>	49
4.3.2	<u>Chat & Messaging – Safe Communication</u>	49
4.3.3	<u>AI Safety Monitoring – Detecting Cyberbullying</u>	49
4.3.4	<u>Alerts & Reports – Parent/Teacher Awareness</u>	49
4.3.5	<u>Feedback & Complaints – Continuous Improvement</u>	49
4.3.6	<u>Subscription & Payment (For Parents)</u>	50
4.3.7	<u>Flow Highlights</u>	50
4.4	<u>Design Models</u>	51
4.4.1	<u>Class Diagram Description</u>	51
4.4.2	<u>Sequence Diagram Description</u>	54

4.4.3	<u>System Diagram</u>	55
5	<u>Implementation</u>	61
5.1	<u>Algorithm</u>	61
5.1.1	<u>AI-Based Message Moderation</u>	61
5.1.2	<u>User Authentication</u>	62
5.1.3	<u>AI-Based Cyberbullying Detection</u>	62
5.1.4	<u>Safe Chat Process</u>	63
5.1.5	<u>Parent/Teacher Alert Generation</u>	64
5.1.6	<u>Abuse Reporting Process</u>	64
5.2	<u>External APIs</u>	65
5.3	<u>User Interface</u>	66
6	<u>Testing and Evaluation</u>	76
6.1	<u>Manual Testing</u>	76
6.1.1	<u>System Testing</u>	76
6.1.2	<u>Unit Testing</u>	77
6.1.3	<u>Functional Testing</u>	81
6.1.4	<u>Integration Testing</u>	85
6.2	<u>Automated Testing</u>	86
6.2.1	<u>Tools Used</u>	87
7	<u>Conclusion and Future Work</u>	91
7.1	<u>Conclusion</u>	91
7.2	<u>Future Work</u>	92
8	<u>References</u>	94

LIST OF FIGURES

Figure 3. 1:Use Case Diagram	20
Figure 4. 1:Architecture Diagram	46
Figure 4. 2:ERD Diagram	46
Figure 4. 3:Admin Activity Diagram.....	47
Figure 4. 4:Child Activity Diagram	47
Figure 4. 5:Parent Activity Diagram.....	48
Figure 4. 6:Teacher Activity Diagram	48
Figure 4. 7:Class Diagram	51
Figure 4. 8: Sequence Diagram.....	54
Figure 4. 9: Child System Diagram	56
Figure 4. 10: Admin System Diagram	57
Figure 4. 11:Parent System Diagram	58
Figure 4. 12: Teacher System Diagram	60
Figure 5. 1:Splash Screen	67
Figure 5. 2:Login Screen.....	68
Figure 5. 3: My Profile.....	69
Figure 5. 4: Child Chat Interface	70
Figure 5. 5: Rewards Page	71
Figure 5. 6:Parent Dashboard	72
Figure 5. 7:Teacher Panel	73
Figure 5. 8:Admin Panel	74
Figure 5. 9:Moderation Action Screen.....	75

LIST OF TABLES

Table 3. 1:Access Education Resources	21
Table 3. 2:Earn Badge.....	22
Table 3. 3:Safe Chat Messaging	23
Table 3. 4: AI Based Detection.....	24
Table 3. 5:Send Notification	25
Table 3. 6:Receive Notification	26
Table 3. 7:Review Flagged Messages.....	27
Table 3. 8:Apply Moderation Action.....	28
Table 3. 9:View Moderation History	29
Table 3. 10:View Moderation History	30
Table 3. 11:Approve Account.....	31
Table 3. 12:Register	32
Table 3. 13:Login.....	33
Table 3. 14:Logout.....	34
Table 3. 15:View Complaint.....	36
Table 3. 16:View Child Progress	36
Table 3. 17:Publish Education Resources.....	37
Table 3. 18:Report Abuse	38
Table 3. 19:Manage Classroom	39
Table 3. 20:Enrollment	40
Table 5. 1:Details of APIs Used in the Project.....	65
Table 6. 1:Login Unit Test Case.....	77
Table 6. 2:Registration Unit Test Case	78
Table 6. 3:AI Moderation Unit Test Case.....	78
Table 6. 4:Reward System Unit Test Case	79
Table 6. 5:Parent Dashboard Unit Test Case	79
Table 6. 6:Teacher Panel Unit Test Case.....	80
Table 6. 7:Firebase Notification Unit Test Case.....	80
Table 6. 8:SignalR Unit Test Case.....	81
Table 6. 9:Login Functional Test Case	81
Table 6. 10: AI Message Classification Functional Test Case	82
Table 6. 11:Child Chat Functional Test Case	82
Table 6. 12:Reward System Functional Test Case	83
Table 6. 13:Parent Dashboard Functional Test Case	83
Table 6. 14:Teacher Panel Functional Test Case.....	84
Table 6. 15:Firebase Notification Functional Test Case.....	84
Table 6. 16:SignalR Functional Test Case.....	85
Table 6. 17:Integration Test Case	85
Table 6. 18:Tools Used for Automated Testing.....	87
Table 6. 19:AI Microservice Test Results	88

Table 6. 20:Authentication Test Results	88
Table 6. 21:Database Test Results	89
Table 6. 22:Performance Test Results	89
Table 6. 23:Security Test Results	90

1. Introduction

The aim of this project is to introduce KidSafe – AI-Based Cyberbullying Detection App, an innovative solution designed to enhance digital safety and support the mental well-being of children in online communication environments. With the rapid growth of digital platforms such as chat applications, social media, and messaging services, children are increasingly exposed to cyberbullying, verbal abuse, and online harassment, often unnoticed by parents and educators, leading to serious emotional and psychological effects.

Existing tools like Bark and Google Family Link rely mainly on keyword-based filtering, which fails to understand context, tone, and sarcasm used in modern cyberbullying. This limitation allows harmful messages to bypass detection, creating a need for a more intelligent and context-aware system.

KidSafe addresses this gap using AI, NLP, and ML, analyzing messages in real time before delivery. It uses a lightweight BiLSTM-based model to detect toxic, sarcastic, or disguised bullying even on low-resource devices. Messages are classified as safe, flagged, or blocked to ensure secure communication.

The system also includes a child-friendly chat interface with emojis, stickers, and reward-based features like badges to encourage positive behavior. Parent and teacher dashboards provide real-time alerts, reports, and analytics via Firebase notifications, while maintaining a privacy-first approach by storing only flagged data.

By combining intelligent content moderation, secure backend design, and technologies like Blazor .NET and ASP.NET Core, KidSafe creates a safe digital environment for children. This report covers the background, motivation, objectives, scope, methodology, and feasibility of the system, highlighting its technical strength and societal impact.

1.1. Brief

This project presents KidSafe AI-Based Cyberbullying Detection App, a smart solution designed to improve children's safety in digital communication by detecting and preventing cyberbullying in real time before messages are delivered.

The system is powered by AI, NLP, and Machine Learning, using a BiLSTM deep learning model to understand context and classify messages as safe or harmful. The model is optimized with TensorFlow Lite for efficient deployment.

It is developed using TensorFlow for training, FastAPI for AI service, and ASP.NET Core with Blazor for frontend and backend integration. The methodology includes data preprocessing, model training, evaluation, and deployment via RESTful API.

This report covers the full development lifecycle, including requirement analysis, system design, implementation, testing, results, limitations, and future improvements, demonstrating an AI-based approach to building a safer online environment for children.

1.2. Relevance to Course Modules

This project integrates concepts from various core modules studied throughout the Bachelor of Science in Computer Science program. It reflects the application of:

Programming Fundamentals (PF): Provided basic programming logic used in implementing data processing and system functionality.

Object-Oriented Programming (OOP): Used to design the system with classes, objects, and modular architecture.

Artificial Intelligence (AI): Applied to develop the cyberbullying detection logic and intelligent decision-making system.

Machine Learning (ML): Used to train the BiLSTM model using supervised learning for text classification.

Natural Language Processing (NLP): Implemented for text preprocessing, tokenization, and feature extraction to understand message context.

Web Development: Applied through ASP.NET Core and Blazor for building the frontend interface and backend services.

API Development: Implemented using FastAPI to connect the AI model with the application via RESTful services.

Database Systems: Used for storing and managing flagged messages and user-related data securely.

Human-Computer Interaction (HCI): Helped design a user-friendly interface suitable for children, parents, and teachers.

Software Engineering: Guided requirement analysis, system design, development process, and documentation.

1.3. Project Background

With the rapid growth of digital communication platforms such as chat apps, social media, and messaging systems, children are increasingly exposed to online interactions at a young age. While these platforms offer learning and social benefits, they also introduce risks like cyberbullying, abusive language, and online harassment, which can cause serious psychological and emotional harm.

Traditional parental control tools rely on keyword-based filtering, which cannot understand context, tone, sarcasm, or language variations, allowing many harmful messages to go undetected and reducing their effectiveness.

Advancements in Artificial Intelligence, especially Natural Language Processing (NLP) and Machine Learning, enable systems to understand language contextually and detect harmful content more accurately.

The KidSafe AI project leverages these technologies to build a smart cyberbullying detection system that analyzes messages in real time and classifies them as safe or harmful. It includes a child-friendly chat interface, secure backend, and monitoring dashboards for parents and teachers, aiming to provide a safe digital environment while maintaining privacy and usability.

1.4. Literature Review

In recent years, various tools and research efforts have addressed online safety and cyberbullying detection. Applications like Bark and Google Family Link provide basic monitoring features such as keyword filtering and activity tracking, but they rely on predefined keywords and fail to understand context, sarcasm, and evolving slang.

In Natural Language Processing (NLP), traditional machine learning models like SVM and Naïve Bayes have been used for text classification but lack deep contextual understanding. Advanced deep learning models such as RNN and LSTM improve performance by capturing sequential patterns in text.

Recent research shows that BiLSTM models perform better by processing text in both directions, enhancing context understanding. Transformer-based models further improve accuracy by capturing complex linguistic and semantic relationships.

Despite these advancements, existing systems face issues such as high computational cost, lack of real-time processing, privacy concerns, and absence of integrated solutions combining detection, interface, and monitoring.

The KidSafe AI project addresses these gaps by using a lightweight BiLSTM model with a FastAPI-based real-time prediction service and ASP.NET Core backend for secure data handling and notifications. It follows a privacy-first approach by storing only flagged messages and provides a complete, efficient solution for safe communication.

1.5. Analysis from Literature Review

The literature review shows that existing cyberbullying detection systems and parental control tools mainly rely on keyword-based filtering, which cannot understand context, tone, sarcasm, or evolving language. As a result, many modern forms of cyberbullying remain undetected, creating a gap in ensuring real-time child safety.

Research highlights that machine learning and deep learning models such as LSTM, BiLSTM, and transformer-based models improve detection by capturing contextual and sequential information. However, these approaches often require high computational

resources, lack real-time deployment, and are not well integrated with user-friendly applications.

The KidSafe AI system addresses these limitations by using a BiLSTM-based model for accurate contextual analysis with efficient real-time performance. It is deployed using a FastAPI service and integrated with an ASP.NET Core backend for secure processing.

Unlike traditional systems, KidSafe not only detects but also classifies messages into toxicity levels and takes actions like blocking or flagging. It includes a child-friendly chat interface and dashboards for parents and teachers, providing alerts while maintaining privacy by storing only flagged messages.

Overall, KidSafe improves existing solutions by combining advanced AI, real-time processing, secure integration, and user-focused design, making it a practical and scalable system for children's online safety.

1.6. Methodology and Software Lifecycle for This Project

The development of the KidSafe AI system follows an Object-Oriented Programming (OOP) paradigm combined with the Incremental Software Development Life Cycle (SDLC). The OOP approach is used to design modular, reusable, and scalable components such as the chat interface, backend services, and AI integration modules, ensuring better code organization and maintainability.

The Incremental model is adopted to develop the system in small functional parts (increments), where each module frontend (Blazor), backend (ASP.NET Core), and AI model (FastAPI with BiLSTM) is built, tested, and integrated step by step. The development phases include requirement analysis, system design, implementation, testing, deployment, and evaluation, ensuring continuous validation and improvement at each stage.

1.6.1. Rationale behind Selected Methodology

The Object-Oriented methodology is selected because the system consists of multiple interacting components such as user interface, API services, and machine learning modules. OOP allows the system to be divided into independent classes and objects, making the design more structured, reusable, and scalable for future enhancements like dashboards and notification systems. The Incremental SDLC model is chosen due to its simplicity and effectiveness in handling complex systems. Since the project involves multiple modules (frontend, backend, and AI model), the incremental approach allows each component to be developed and tested separately before integration. It supports early detection of errors, better risk management, and gradual system improvement, making it suitable for building a reliable real-time cyberbullying detection system.

2. Problem Definition

With the increasing use of digital communication platforms, children are more vulnerable to cyberbullying, online harassment, and harmful content. Existing parental control systems rely on keyword-based filtering, which fails to understand context, tone, sarcasm, and evolving language, allowing many harmful messages to go undetected.

The core problem is the absence of an intelligent, real-time system that can accurately detect and prevent cyberbullying while ensuring privacy and usability. Current solutions also lack integration of features like user-friendly communication platforms, real-time alerts for parents/teachers, and secure data handling, creating a gap between detection, response, and monitoring.

The outcome of this project is KidSafe AI, an AI-based system that analyzes messages in real time using a machine learning model, classifies them as safe or harmful, and blocks or flags inappropriate content. It provides alerts and reports through a monitoring dashboard and maintains privacy by storing only flagged messages, aiming to create a safer digital environment for children.

2.1. Problem Statement

Children's increasing use of digital communication platforms has exposed them to cyberbullying, abusive language, and harmful interactions that often go undetected. Existing monitoring systems mainly rely on keyword-based filtering, which lacks the ability to understand context, tone, and variations in language, resulting in inaccurate detection of harmful messages.

There is a need for an intelligent system that can analyze chat messages in real time, accurately identify cyberbullying using context-aware techniques, and take immediate action to prevent harmful content from reaching children. Additionally, the system must provide monitoring capabilities for parents and teachers while ensuring user privacy through secure data handling and minimal data storage.

Therefore, the problem is to design and develop an AI-based solution that integrates machine learning, real-time message filtering, and a secure application environment to effectively detect, block, and report cyberbullying in children's digital communication.

2.2. Deliverables and Development Requirements

The following points show key Deliverables and Development requirements:

2.2.1 Deliverables:

A mobile and desktop-based, child-friendly chat application (KidSafe AI). AI-based cyberbullying detection model (BiLSTM). Real-time message filtering and blocking system. Parent and teacher monitoring dashboard with alerts.

FastAPI-based AI prediction service. Secure backend system with notification and data handling. Privacy-focused storage system (flagged messages only)

2.2.2 Development Requirements:

Frontend developed using Blazor (.NET MAUI). Backend developed using ASP.NET Core. AI model built using TensorFlow (BiLSTM). FastAPI for AI model deployment and API services. Firebase for notifications and real-time alerts. HTTP/REST API integration between frontend, backend, and AI service. Text dataset for cyberbullying detection (training and testing). Internet connectivity for real-time communication and API calls

3. Requirement Analysis

This chapter identifies and analyses the functional and non-functional requirements of the KidSafe AI-Based Cyberbullying Detection System. Requirement analysis is one of the most important phases of software development because it defines the expected behaviour of the system and ensures that the final application satisfies the needs of its intended users. It provides a clear understanding of the system's features, performance expectations, security requirements, and operational constraints before implementation begins.

The requirements presented in this chapter are derived from the project objectives, the Software Requirements Specification (SRS) document, and the overall system design. These requirements ensure that every component of the system works together efficiently to achieve accurate cyberbullying detection while maintaining user privacy and system reliability.

The KidSafe system is designed to provide real-time message analysis by continuously monitoring user-generated text and identifying potentially harmful or offensive content. Artificial Intelligence, specifically the Bidirectional Long Short-Term Memory (BiLSTM) model, is utilized to perform toxicity detection by analysing the context and sequence of words within a message. This enables the system to classify messages with improved accuracy and reduce false detections.

In addition, the system incorporates secure backend processing through FastAPI and ASP.NET Core, allowing efficient communication between different system components while ensuring scalability and high performance. The backend architecture supports secure data processing, user authentication, API communication, and the management of moderation activities. Furthermore, the system follows a privacy-focused data handling approach by processing sensitive user information securely and minimizing unnecessary data storage, thereby protecting user confidentiality and complying with modern software security practices.

Overall, this chapter provides a detailed overview of the system requirements that serve as the foundation for the design, implementation, and successful operation of the KidSafe AI-Based Cyberbullying Detection System.

3.1. Use Cases Diagram

The following figure 3.1 shows the Use case diagram of the KidSafe AI-Based Cyberbullying Detection System illustrates the interaction between the different users and the system. It provides a high-level view of the system's functionality by identifying the primary actors and the tasks they can perform within the application. The diagram demonstrates how each user interacts with various system features based on their assigned roles and responsibilities.

.

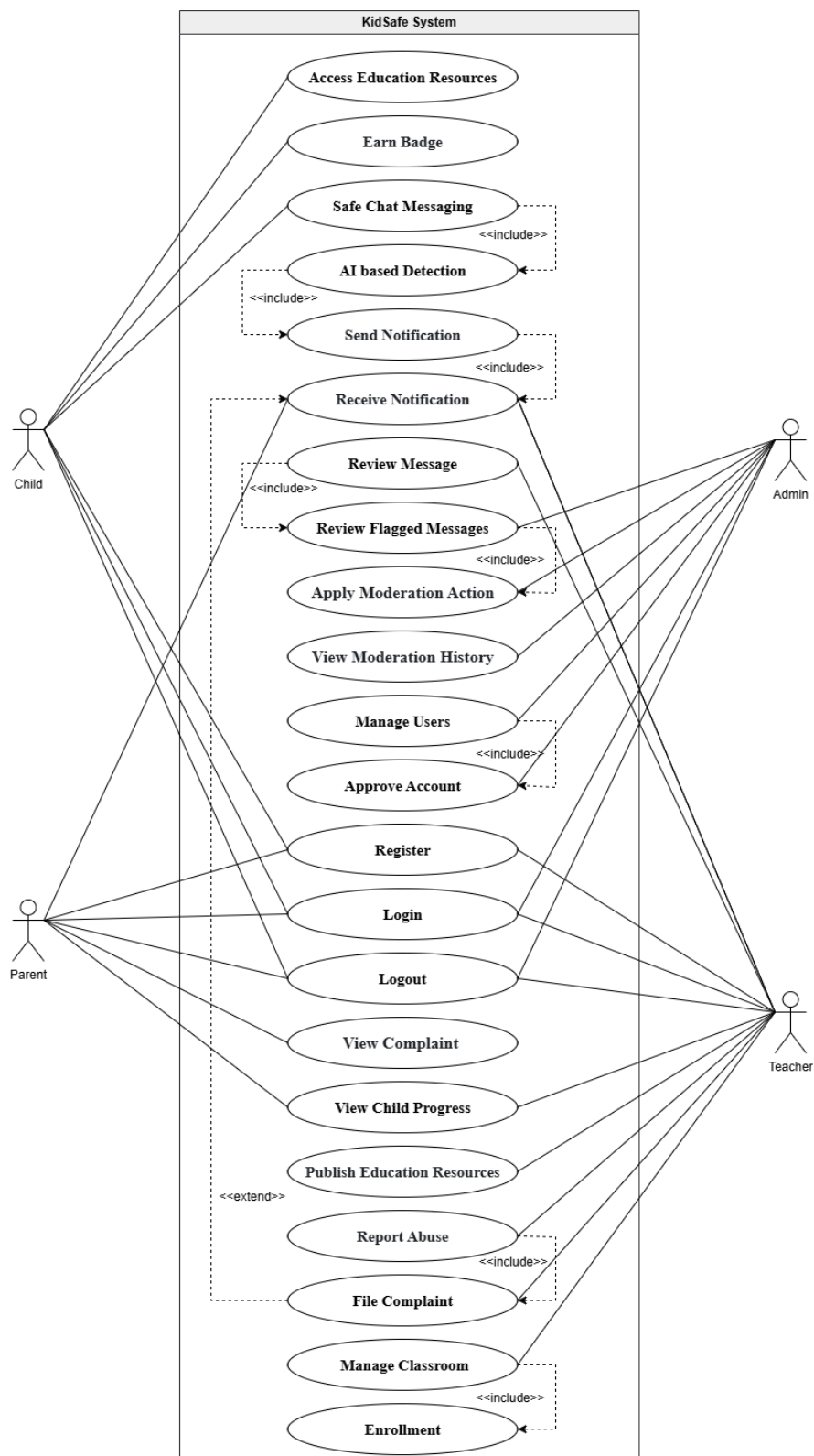


Figure 3. 1: Use Case Diagram

3.1.1 Use Case Description

This section describes the main actors of the KidSafe system and their interactions with the AI-based cyberbullying detection platform. Each actor plays a specific role in ensuring safe communication and effective monitoring of online interactions.

The child is the primary user of the chat system who sends and receives messages. All messages are monitored in real time to ensure a safe and positive communication environment.

The parent monitors the child's online activity through a dedicated dashboard. They receive alerts and can review flagged or harmful messages for safety assurance.

The teacher observes student behavior through system-generated reports and notifications. This helps in identifying cyberbullying cases and providing timely support.

The admin manages the overall system operations, user accounts, and security. They also monitor system logs and ensure smooth functioning of the AI detection service.

The BiLSTM model processes chat messages using deep learning and contextual understanding. It classifies messages as safe or harmful and provides probability scores for real-time decision-making.

3.2. Detailed Use Case

The following section provides comprehensive information about all the identified use cases.

The following table 3.1 shows the details of the UC-01 "Access Education Resources" use case.

Table 3. 1:Access Education Resources

Field	Explanation & Example
Use Case ID	UC-01
Use Case Name	Access Education Resources

Actors	Primary Actor: Child Secondary Actors: AI Service (search), Notification System
Description	A child accesses learning and online safety materials, views them, optionally takes a quick quiz, and marks completion.
Trigger	Child opens Learn or selects a recommended resource.
Preconditions	PRE-1. User authenticated PRE-2. Role permitted to view resources
Postconditions	POST-1. Resource viewed POST-2. Progress/badge counters optionally updated
Normal Flow	1. Open Learn 2. Browse/Continue 3. Open resource 4. Read/watch 5. Optional quick quiz 6. Mark complete
Alternative Flows	AF-1. Offline mode → show cached materials AF-2. Safe search returns no results → show kid-friendly explanation
Exceptions	EX-1. Resource unavailable → show retry/download prompt
Business Rules	BR-1. Age-appropriate filtering on resources BR-2. Quiz scoring thresholds configured by admin
Assumptions	A-1. Users often resume previously started resources

The following table 3.2 shows the details of the UC-02 " Earn Badge " use case.

Table 3. 2: Earn Badge

Field	Explanation & Example
Use Case ID	UC-02
Use Case Name	Earn Badge

Actors	Primary Actor: Child Secondary Actors: Badge Service, Notification System
Description	Child earns badges/XP upon completing tasks or displaying positive behavior; system updates level and may celebrate.
Trigger	Completion of a qualifying task or behavior event.
Preconditions	PRE-1. Badge rules configured PRE-2. Awardable actions available
Postconditions	POST-1. Badge awarded POST-2. XP/level updated POST-3. Celebration shown
Normal Flow	1. Complete task 2. System evaluates criteria 3. Award badge 4. Show badge detail 5. Optional share (if permitted)
Alternative Flows	AF-1. Offline completion → queue award AF-2. Duplicate detection prevents re-award
Exceptions	EX-1. Badge rule conflict → log and skip award
Business Rules	BR-1. One-time badges cannot be re-awarded BR-2. XP scales with task difficulty
Assumptions	A-1. Some awards are delayed by moderation checks

The following table 3.3 shows the details of the UC-03 " Safe Chat Messaging " use case.

Table 3. 3:Safe Chat Messaging

Field	Explanation & Example
Use Case ID	UC-03
Use Case Name	Safe Chat Messaging

Actors	Primary Actor: Child Secondary Actors: AI Detection (UC 04), Notification System, Guardians/Teachers (alerts)
Description	Child sends/receives messages screened for safety; safe messages are delivered, flagged content handled securely.
Trigger	Child composes and sends a message.
Preconditions	PRE-1. Authenticated PRE-2. Recipient permitted PRE-3. AI service reachable
Postconditions	POST-1. Safe → delivered POST-2. Flagged/blocked → encrypted; optional guardian/teacher alert
Normal Flow	1. Open Chat 2. Compose 3. Send 4. UC 04 evaluates 5. Deliver if safe or show nudge to edit/report
Alternative Flows	AF-1. AI unreachable → fallback heuristic/queue AF-2. DND enabled → queue notifications
Exceptions	EX-1. Recipient not permitted → block and explain
Business Rules	BR-1. Safety check mandatory pre-delivery BR-2. Retention policy limits storage of safe messages
Assumptions	A-1. Network latency can delay delivery feedback

The following table 3.4 shows the details of the UC-04 " AI Based Detection " use case.

Table 3. 4: AI Based Detection

Field	Explanation & Example
Use Case ID	UC-04
Use Case Name	AI Based Detection

Actors	Primary Actor: AI System Secondary Actors: Moderation System, Notification System
Description	Classifies messages/media/links and returns labels and kid-friendly nudges; updates encrypted safety logs.
Trigger	New content submitted from Chat or other entry points.
Preconditions	PRE-1. AI system available PRE-2. Secure keys valid PRE-3. Input sanitized
Postconditions	POST-1. Classification returned POST-2. Encrypted safety logs updated
Normal Flow	1. Receive sanitized input 2. Score/label 3. Return result + nudge 4. Optionally trigger alerts
Alternative Flows	AF-1. Low confidence → auto-flag AF-2. Model fallback AF-3. Rate limit reached → queue
Exceptions	EX-1. Invalid payload → drop and log
Business Rules	BR-1. PII redacted before inference BR-2. Toxic content stored encrypted only
Assumptions	A-1. Models periodically updated without downtime

The following table 3.5 shows the details of the UC-05 " Send Notification " use case.

Table 3. 5:Send Notification

Field	Explanation & Example
Use Case ID	UC-05
Use Case Name	Send Notification

Actors	Primary Actor: System (auto), Admin/Teacher (manual) Secondary Actors: Push/Web Delivery Service
Description	Compose and dispatch notifications for learning reminders, system updates, or safety alerts.
Trigger	Scheduled or event-driven notification request.
Preconditions	PRE-1. Recipient list resolved PRE-2. Device tokens/web endpoints valid
Postconditions	POST-1. Notification queued/dispatched POST-2. Delivery attempts logged
Normal Flow	1. Compose payload (title, brief, target) 2. Validate 3. Schedule/send 4. Record status
Alternative Flows	AF-1. Invalid targets → skip with error AF-2. Rate limit/batch send
Exceptions	EX-1. Delivery service unavailable → retry/backoff
Business Rules	BR-1. Quiet hours honored by role BR-2. Safety content never in plaintext
Assumptions	A-1. Multi-device recipients may receive duplicates (deduplication in place)

The following table 3.6 shows the details of the UC-06 "Receive Notification" use case.

Table 3. 6:Receive Notification

Field	Explanation & Example
Use Case ID	UC-06
Use Case Name	Receive Notification

Actors	Primary Actor: Child, Parent, Teacher, Admin Secondary Actors: Push/Web Delivery Service
Description	Users receive and open notifications; system records acknowledgment.
Trigger	Incoming push/web notification.
Preconditions	PRE-1. User linked PRE-2. Device/browser registered PRE-3. Permissions granted
Postconditions	POST-1. Alert delivered/opened POST-2. Acknowledgment recorded
Normal Flow	1. Deliver via push/web 2. User opens 3. Deep link to feature 4. Mark as read
Alternative Flows	AF-1. Offline/quiet hours → queued/delayed delivery
Exceptions	EX-1. Token expired → re-register prompt
Business Rules	BR-1. Minimal context only BR-2. No plain text harmful content
Assumptions	A-1. Some clients defer delivery due to OS policies

The following table 3.7 shows the details of the UC-07 " Review Flagged Messages " use case.

Table 3. 7:Review Flagged Messages

Field	Explanation & Example
Use Case ID	UC-07
Use Case Name	Review Flagged Messages
Actors	Primary Actor: Parent, Teacher, Admin Secondary Actors: Moderation System

Description	Review AI-flagged or blocked items, acknowledge or escalate, and add notes.
Trigger	Flagged items appear in review queue.
Preconditions	PRE-1. Flagged items exist PRE-2. Role permitted
Postconditions	POST-1. Reviewed/acknowledged POST-2. Optional resolution notes saved
Normal Flow	1. Open flagged list 2. Inspect details/rationale 3. Acknowledge/comment 4. Escalate if needed
Alternative Flows	AF-1. False positive → route to UC 09 or feedback
Exceptions	EX-1. Item expired → archive
Business Rules	BR-1. Access to encrypted content only BR-2. Review SLAs by severity
Assumptions	A-1. Reviewers may batch-process items

The following table 3.8 shows the details of the UC-08 " Apply Moderation Action " use case.

Table 3. 8:Apply Moderation Action

Field	Explanation & Example
Use Case ID	UC-08
Use Case Name	Apply Moderation Action
Actors	Primary Actor: Admin (full), Teacher (limited) Secondary Actors: Notification System

Description	Take actions on flagged content/users (mute, block, warn, remove content) and log outcomes.
Trigger	Action initiated from a flagged case.
Preconditions	PRE-1. Flagged case exists PRE-2. Authority granted
Postconditions	POST-1. Action applied POST-2. Participants notified (if applicable) POST-3. Audit updated
Normal Flow	1. Open case 2. Choose action 3. Confirm 4. Log outcome
Alternative Flows	AF-1. Revoke/undo within window AF-2. Escalate to Admin
Exceptions	EX-1. Policy conflict → require Admin approval
Business Rules	BR-1. Progressive discipline policy BR-2. Audit entries immutable
Assumptions	A-1. Some actions impact chat group membership immediately

The following table 3.9 shows the details of the UC-9 " View Moderation History " use case.

Table 3. 9:View Moderation History

Field	Explanation & Example
Use Case ID	UC-9
Use Case Name	View Moderation History
Actors	Primary Actor: Admin Secondary Actors: Audit Service
Description	Admin views or exports chronological record of moderation actions.

Trigger	Admin opens Moderation History.
Preconditions	PRE-1. Audit data exists
Postconditions	POST-1. History viewed/exported
Normal Flow	1. Filter by user/date/action 2. Open record 3. Export if needed
Alternative Flows	AF-1. No records → empty view
Exceptions	EX-1. Export limit exceeded → paginate
Business Rules	BR-1. Minimal PII in exports BR-2. Access restricted to Admin
Assumptions	A-1. Exports used for compliance reviews

The following table 3.10 shows the details of the UC-10 " Manage Users " use case.

Table 3. 10:View Moderation History

Field	Explanation & Example
Use Case ID	UC-10
Use Case Name	Manage Users
Actors	Primary Actor: Admin Secondary Actors: Audit Service
Description	Admin searches and updates user roles/settings; changes are audited.
Trigger	Admin initiates user management.
Preconditions	PRE-1. Admin authenticated
Postconditions	POST-1. Updates stored POST-2. Audit updated

Normal Flow	1. Search user 2. Modify roles/settings 3. Save 4. Confirm
Alternative Flows	AF-1. Revoke role → validation checks AF-2. Bulk updates
Exceptions	EX-1. Permission conflict → reject change
Business Rules	BR-1. Least privilege enforcement BR-2. Role changes logged
Assumptions	A-1. Some changes require re-login

The following table 3.11 shows the details of the UC-11 " Approve Account " use case.

Table 3. 11:Approve Account

Field	Explanation & Example
Use Case ID	UC-11
Use Case Name	Approve Account
Actors	Primary Actor: Admin Secondary Actors: Notification System
Description	Admin reviews pending accounts and approves or rejects with comments.
Trigger	Pending account appears in queue.
Preconditions	PRE-1. Pending accounts exist
Postconditions	POST-1. Decision logged POST-2. User notified
Normal Flow	1. Review info 2. Approve/reject 3. Add comment

	4. Finalize
Alternative Flows	AF-1. Bulk approval AF-2. Request more info
Exceptions	EX-1. Missing required verification → reject with reason
Business Rules	BR-1. KYC/age gating rules apply BR-2. Approval SLA targets
Assumptions	A-1. Some accounts auto-approve based on trusted sources

The following table 3.12 shows the details of the UC-12 " Register " use case.

Table 3. 12: Register

Field	Explanation & Example
Use Case ID	UC-12
Use Case Name	Register
Actors	Primary Actor: Child (with Guardian), Parent, Teacher, Admin Secondary Actors: Email/OTP Service
Description	Users create new accounts with verification and consent; minors link with guardians.
Trigger	User opens registration form.
Preconditions	PRE-1. Registration enabled PRE-2. Age gating in place
Postconditions	POST-1. Account created POST-2. Pending UC 12 approval for some
Normal Flow	1. Fill form 2. Verify email/OTP
	3. Accept terms 4. Submit

Alternative Flows	AF-1. Child must link with guardian AF-2. Join via class code first (provisional)
Exceptions	EX-1. Duplicate account → merge or block
Business Rules	BR-1. Consent required for minors BR-2. Minimal PII retention
Assumptions	A-1. Class codes distributed by teachers securely

The following table 3.13 shows the details of the UC-13 " Login " use case.

Table 3. 13:Login

Field	Explanation & Example
Use Case ID	UC-13
Use Case Name	Login
Actors	Primary Actor: All roles Secondary Actors: SSO/MFA Services
Description	Start authenticated user session.
Trigger	User submits credentials.
Preconditions	PRE-1. Valid credentials/SSO
Postconditions	POST-1. Session active POST-2. Redirected to role dashboard
Normal Flow	1. Enter credentials 2. Optional MFA 3. Login success
Alternative Flows	AF-1. Invalid credentials → retry/lockout AF-2. Passwordless via guardian link
Exceptions	EX-1. Account locked → recovery flow

Business Rules	BR-1. Sensitive data not logged BR-2. Session timeout configured
Assumptions	A-1. Some devices use biometric MFA

The following table 3.14 shows the details of the UC-14 "Logout" use case.

Table 3. 14:Logout

Field	Explanation & Example
Use Case ID	UC-14
Use Case Name	Logout
Actors	Primary Actor: All roles
Description	End authenticated session safely and clear local cache.
Trigger	User taps Logout or auto-timeout.
Preconditions	PRE-1. User authenticated
Postconditions	POST-1. Session terminated POST-2. Local cache cleared
Normal Flow	1. Tap Logout 2. Confirm 3. Return to login
Alternative Flows	AF-1. Auto logout after inactivity
Exceptions	EX-1. Pending unsaved work → prompt to save
Business Rules	BR-1. Tokens revoked on logout
Assumptions	A-1. Shared devices require explicit logout reminders

The following table 3.15 shows the details of the UC-15 " View Complaint " use case.

Table 3. 15:View Complaint

Field	Explanation & Example
Use Case ID	UC-15
Use Case Name	View Complaint
Actors	Primary Actor: Child (own items), Parent, Teacher, Admin
Description	View complaint status and details with role-based access.
Trigger	User opens Support → Complaints.
Preconditions	PRE-1. At least one complaint exists PRE-2. Role authorized
Postconditions	POST-1. Status viewed POST-2. Optional acknowledgment added
Normal Flow	1. Open Support 2. My Complaints/All (role-based) 3. Select item 4. View timeline/notes
Alternative Flows	AF-1. No items → empty state with help
Exceptions	EX-1. Access to unrelated PII blocked
Business Rules	BR-1. Role-based redaction applied
Assumptions	A-1. Users check status after notifications

The following table 3.16 shows the details of the UC-16 " View Child Progress " use case.

Table 3. 16:View Child Progress

Field	Explanation & Example
Use Case ID	UC-16
Use Case Name	View Child Progress

Actors	Primary Actor: Child (self), Parent, Teacher
Description	View learning progress, streaks, and suggestions; may start recommended resources.
Trigger	User opens Progress dashboard.
Preconditions	PRE-1. Progress data exists
Postconditions	POST-1. Insights viewed POST-2. May start recommended resource
Normal Flow	1. Open Progress 2. View charts/streaks 3. Tap suggestion 4. Start UC 01
Alternative Flows	AF-1. No data → show getting started guidance
Exceptions	EX-1. Data fetch failure → retry
Business Rules	BR-1. Aggregated analytics only BR-2. No chat data in insights
Assumptions	A-1. Progress updates nightly

The following table 3.17 shows the details of the UC-17 " Publish Education Resources " use case.

Table 3. 17:Publish Education Resources

Field	Explanation & Example
Use Case ID	UC-17
Use Case Name	Publish Education Resources
Actors	Primary Actor: Teacher, Admin
Description	Create/edit resources, set metadata/audience, and publish to target

	classes/roles.
Trigger	Teacher/Admin opens Authoring.
Preconditions	PRE-1. Authenticated PRE-2. Publish rights granted
Postconditions	POST-1. Resource visible to target classes/roles
Normal Flow	1. Create/edit resource 2. Set metadata/audience 3. Save/publish
Alternative Flows	AF-1. Schedule publish AF-2. Retract/unpublish
Exceptions	EX-1. Policy violation → block publish
Business Rules	BR-1. No student PII in content BR-2. Metadata only stored
Assumptions	A-1. Resources versioned for audit

The following table 3.18 shows the details of the UC-18 " Report Abuse " use case.

Table 3. 18:Report Abuse

Field	Explanation & Example
Use Case ID	UC-18
Use Case Name	Report Abuse
Actors	Primary Actor: Child, Parent, Teacher
Description	Submit reports of harmful messages or behavior, with references.
Trigger	User opens report form or quick report in message.
Preconditions	PRE-1. Authenticated (anonymous allowed with limits) PRE-2. Optional reference to message/user

Postconditions	POST-1. Report recorded POST-2. Reviewer notified
Normal Flow	1. Select reason 2. Add notes/screenshot (optional) 3. Submit 4. Receive ID
Alternative Flows	AF-1. Quick report from message AF-2. Anonymous allowed with limited metadata
Exceptions	EX-1. Insufficient detail → prompt for minimum fields
Business Rules	BR-1. Harmful content linked encrypted BR-2. Reporter privacy protected
Assumptions	A-1. Reports may auto-create from UC 04 outcomes

The following table 3.19 shows the details of the UC-19 " Manage Classroom " use case.

Table 3. 19:Manage Classroom

Field	Explanation & Example
Use Case ID	UC-19
Use Case Name	Manage Classroom
Actors	Primary Actor: Teacher (owner), Admin (oversight)
Description	Create/manage classes, rosters, and chat/resource permissions
Trigger	Teacher opens Classroom management.
Preconditions	PRE-1. Teacher authenticated PRE-2. Ownership or permission
Postconditions	POST-1. Class configuration updated POST-2. Members informed

Normal Flow	1. Create/edit class 2. Manage roster (add/remove) 3. Set permissions 4. Save
Alternative Flows	AF-1. Import roster via CSV/SIS AF-2. Archive class
Exceptions	EX-1. Roster conflicts → resolve prompts
Business Rules	BR-1. Visibility limited to class BR-2. Necessary roster data only
Assumptions	A-1. Changes propagate to chat and resource access promptly

The following table 3.20 shows the details of the UC-20 "Enrollment" use case.

Table 3. 20:Enrollment

Field	Explanation & Example
Use Case ID	UC-20
Use Case Name	Enrollment
Actors	Primary Actor: Child (self join), Teacher (invite)
Description	Join classes via code/invite and view enrolled classes; update chat groups accordingly.
Trigger	User enters code or accepts invite.
Preconditions	PRE-1. Valid class code or invitation PRE-2. Eligibility checks pass
Postconditions	POST-1. Enrollment recorded POST-2. Class appears in dashboard POST-3. Chat groups updated

Normal Flow	1. Open My Classes 2. Enter/join with code or accept invite 3. Confirm 4. See class card/resources
Alternative Flows	AF-1. Invalid/expired code → error with retry AF-2. Request to leave → pending approval
Exceptions	EX-1. Capacity reached → waitlist
Business Rules	BR-1. Membership visible to roster only BR-2. No extra PII shared
Assumptions	A-1. Invites expire after configured time

3.3. Functional Requirements

The following section presents all non-functional requirements in detail.

3.3.1 User Management Module

Handles secure user registration and login for child, parent, teacher, and admin roles. It implements role-based access control to manage permissions and includes account verification and age restrictions to ensure safe and appropriate system usage.

3.3.2 Chat and Messaging Module

Provides real-time messaging with support for text and emojis in a child-friendly interface. It integrates AI-based filtering to detect harmful content before delivery and includes safety features such as block, report, mute, and quiet hours to control interactions.

3.3.3 Cyberbullying Detection and Warnings Module

Uses an AI-based BiLSTM model to analyze messages in real time and classify them as safe or harmful. Based on detection, the system can block or flag messages and generate alerts while maintaining a history of flagged incidents for monitoring purposes.

3.3.4 Parental/Teacher Dashboard

Provides a monitoring platform for parents and teachers to view flagged messages, receive real-time alerts, and analyze children's communication behavior. It also offers reports and insights to support better supervision and decision-making.

3.3.5 Emergency Alert System

Enables children to send instant SOS alerts to parents or teachers in critical situations. The system can also automatically trigger alerts when severe harmful interactions are detected, ensuring quick response and support.

3.3.6 Educational and Support Module

Offers educational content related to mental health, online safety, and digital etiquette. It promotes positive behavior through guidance, awareness material, and supportive resources for children, parents, and teachers.

3.3.7 Notifications Module

Manages system-generated alerts, reminders, and announcements. It includes features like reading, tracking, acknowledgment, and delayed notifications during quiet hours to ensure effective but non-intrusive communication.

3.3.8 Achievement and Progress Module

Tracks user engagement through badges, rewards, and progress indicators. It encourages positive and respectful communication by recognizing good behavior and maintaining motivation among users.

3.3.9 Classroom and Enrollment Module

Allows teachers to create and manage classrooms and enroll students using codes or invitations. It controls access to communication and educational resources based on enrollment status.

3.3.10 Moderation Module

Supports reviewing and managing flagged content by allowing actions such as warning, muting, blocking, or removing users. It maintains secure audit logs and ensures proper enforcement of rules and policies.

3.3.11 Feedback and Complaint Module

Provides a system for users to submit feedback and report issues or complaints. It includes tracking mechanisms to monitor the status of complaints and ensures transparency in handling user concerns.

3.3.12 Educational Resources Module

Enables teachers and administrators to create, manage, and publish educational materials. It ensures that all content is age-appropriate and accessible to relevant users, supporting learning and awareness.

3.4 Non-Functional Requirements

The following section presents all non-functional requirements in detail.

3.4.1 Performance

Response time on chat filters and alerts should be less than 2 seconds. Support up to 1000 online users without performance deterioration. Minimize the inference time of AI models to less than 800 milliseconds of classification response time.

3.4.2 Security

Encrypt sensitive data with the AES 256 algorithm. Make sure that all communication is made under HTTPS. Two-factor authentication of parent and teacher accounts. Keep regular security reviews and penetration testing.

3.4.3 Usability

Offer a child-friendly and user-friendly interface. Make sure that you can perform responsively on desktops, tablets and smartphones. Add support of different languages to support different users. Meet the children with disabilities accessibility requirements.

3.4.4 Scalability

Develop the system to accommodate the additional functions later like voice and video chat surveillance. Make sure that the system will be able to accommodate more users as time goes by.

3.4.5 Reliability

Hit 99.9 percent of continuous monitoring. Automate backups to avoid the loss of data. Offer disaster recovery systems to recover services in time when failed.

3.4.6 Compliance

The KidSafe/KinderNet platform must be able to meet COPPA, GDPR, and other laws on protecting children.

4. Design and Architecture

The following parts of Software Design Description (SDD) report should be included in this chapter.

4.1 System Architecture

KidSafe uses a modular layered architecture, ensuring each part of the system has clear roles and responsibilities.

4.1.1 User Interface Layer:

Manages all user interactions across dashboards, chat screens, and alert views.

- Clients (Blazor .NET MAUI)
- Child App
- Parent Dashboard
- Teacher Dashboard
- Admin Panel
- All clients communicate with backend services via REST APIs.

4.1.2 Application Logic Layer

Implements core business logic and exposes REST endpoints.

- ASP.NET Core Web API manages authentication, roles, reporting, alerts
- AI communication, and workflows.
- Uses Entity Framework Core + SQL (SQLite) for data operations.
- Connects to the Integration Layer for AI checks and push notifications.

4.1.3 Data Storage Layer

Provides secure, structured storage for system data.

- SQLite database managed via Entity Framework Core
- Stores user accounts, chat sessions, flagged messages, reports, complaints, feedback, badges, enrollment records, and audit logs
- Enforces encryption at rest and audit trails

4.1.4 Modules and Relationships User Management

Secure account creation and login for Children, Parents, Teachers, and Admins

- Blazor frontend, ASP.NET API, SQLite storage

4.1.5 Chat & Communication

- Real-time messaging with AI checks before delivery
- ASP.NET Core API, AI Service, Send / Blur / Block message
- Messages stored in SQLite

4.1.6 AI Safety Filter

NLP/ML models (e.g., BiLSTM) detect harassment, threats, and toxic content

- Implemented as a Python FastAPI microservice
- Returns labels like Safe / Watch / Review

4.1.7 Alerts & Notifications

- Real-time push notifications via Firebase
- Triggered automatically when unsafe content is detected
- Supports queued delivery and read-status tracking

4.1.8 Report Management

- Children can report issues
- Parents/teachers review incidents
- Linked flagged messages stored securely

4.1.9 Emergency Support

- Quick SOS action for severe bullying
- Alerts routed via API + Firebase
- Provides immediate guidance in the Child App

4.1.10 Consultation & Support System

- Scheduling and contacting counselors/teachers
- Orchestrated by ASP.NET Core
- Metadata stored in SQLite

4.1.11 Document & Media Management

- Stores evidence such as screenshots or complaint files
- Controlled uploads and secure storage

4.1.12 Feedback & Complaint System

- Parents/teachers share system feedback
- Admins manage lifecycle and status updates
- Notifications sent for changes or resolutions

The Following Figure 4.1 illustrates the Layered Architecture of the KidSafe system

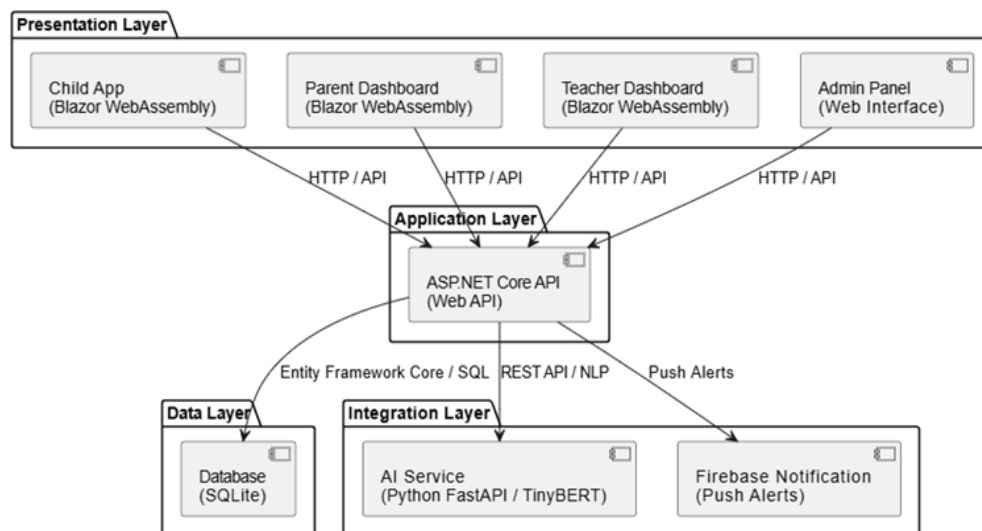


Figure 4. 1:Architecture Diagram

4.2 Data Representation

This section defines the database structure and entity relationships used in the system. The presented ERD illustrates the database structure of the KidSafe – AI-Based Cyberbullying Detection System, showing how different entities interact to support communication, monitoring, and safety enforcement.

The Users entity represents all participants in the system, including children, parents, teachers, and admins. Each user has basic details like ID, name, email, password, role, and timestamps. Users interact with the system by sending messages, managing classrooms, and accessing features based on their role.

Users exchange messages, which are analyzed by the moderation system. Each message includes sender/receiver details and content. The system assigns a status (Safe, Flagged,

Blocked) and a toxicity score (0–1). Harmful messages are flagged, stored, and reviewed by teachers or admins, ensuring privacy by saving only necessary data.

The AI model processes messages to detect cyberbullying. It generates predictions such as toxicity score and classification results, which directly influence moderation actions like flagging or blocking content.

The system includes a dedicated Children entity. Kids can participate in chats and earn badges and achievements for positive behavior, encouraging respectful communication.

Teachers or admins create classrooms to organize children into groups. Enrollments link students to classrooms, and educational resources can be shared within these groups.

The system sends notifications to parents and teachers when harmful activity is detected or reports are generated, enabling real-time monitoring and quick action.

4.2.1 Relationships Overview

- A User can send and receive multiple Messages (One-to-Many)
- A Message can result in one Moderation record
- A Moderation can trigger Flagged Messages
- A User can create Classrooms
- Children enroll in classrooms via Enrollments
- Classrooms include multiple children and resources
- Notifications are linked to users and events
- Reports and Reviews support administrative control

The Following Figure 4.2 illustrates the Entity Relationship diagram of the KidSafe System, as shown below.

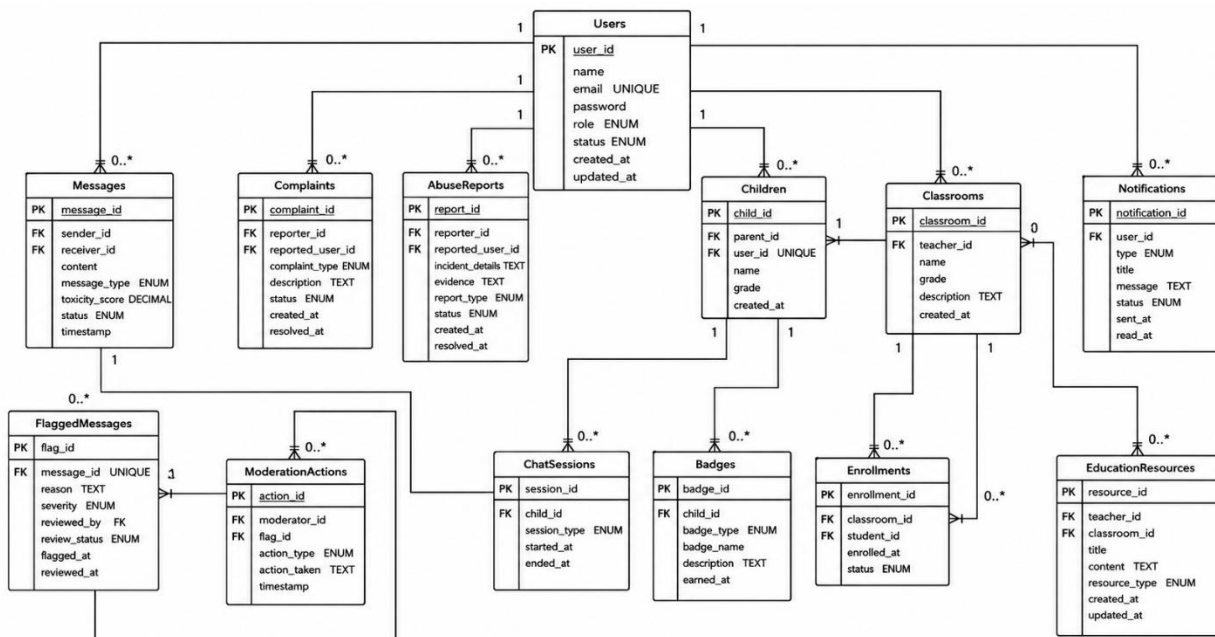


Figure 4. 2:ERD Diagram

4.3 Process Flow/Representation

The KidSafe system provides secure user authentication through a login and account management mechanism for four user roles: Child, Parent, Teacher, and Admin. It enables safe communication by allowing child users to send and receive chat messages, which are analysed by the AI Safety Filter before delivery to ensure a secure chatting environment. The AI Bullying Detection module analyses text messages in real time, blocks harmful content, and automatically sends alerts to Parents and Teachers when inappropriate behaviour is detected. In addition, the system supports report and complaint handling, allowing children to report bullying incidents, while Parents and Teachers can review these reports and submit complaints when necessary. The application also generates automatic alerts and notifications whenever bullying is detected or reported. Furthermore, KidSafe provides emergency guidance by offering children appropriate safety steps if they feel threatened online. It also includes consultation and support features that enable Parents and Children to connect with Teachers or the Admin for further assistance and counselling.

The Following Figure 4.3 illustrates the Activity Diagram of Admin, as shown below.

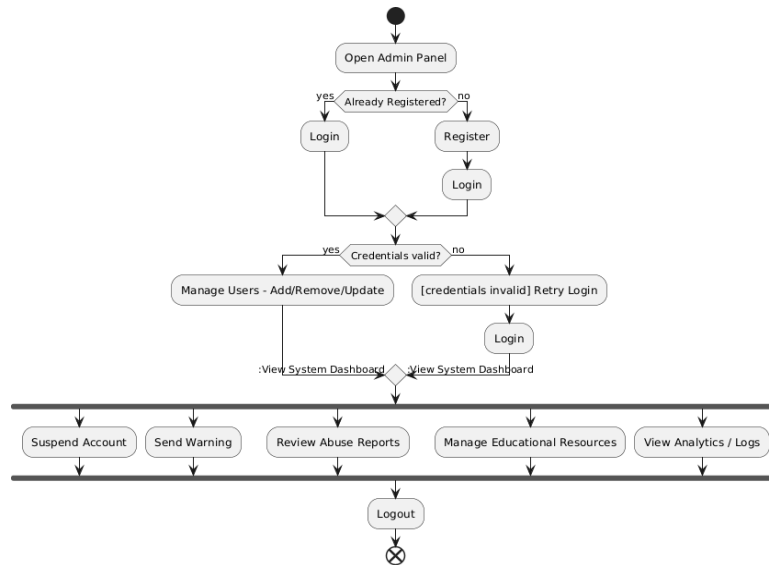


Figure 4. 3:Admin Activity Diagram

The Following Figure 4.4 illustrates the Activity Diagram of Child, as shown below.

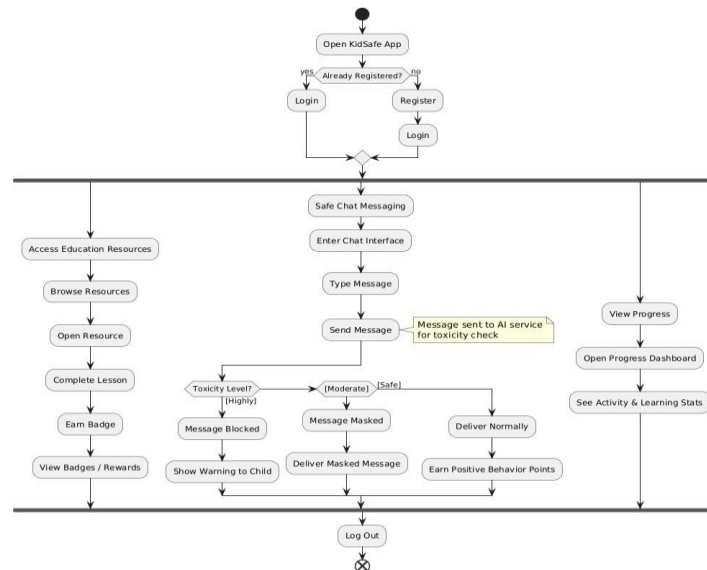


Figure 4. 4:Child Activity Diagram

The Following Figure 4.5 illustrates the Activity Diagram of Parent, as shown below.

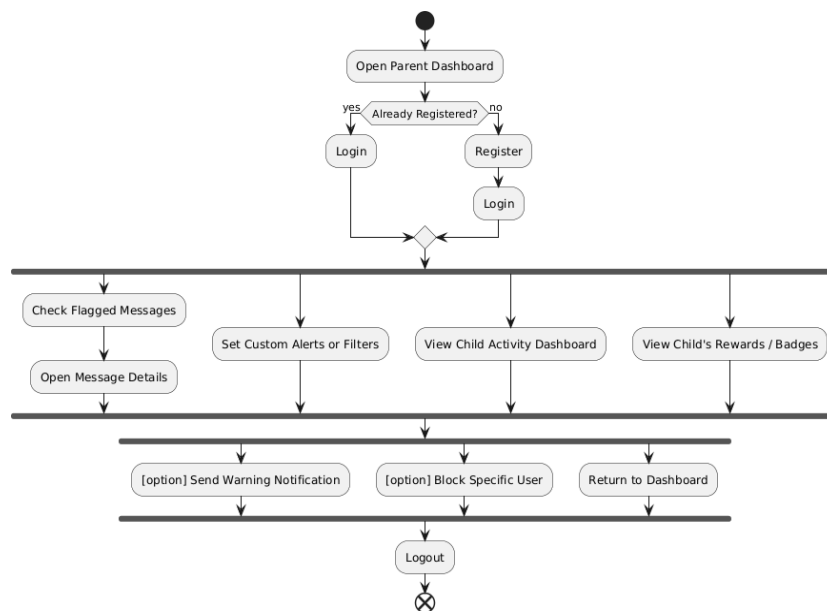


Figure 4. 5:Parent Activity Diagram

The Following Figure 4.6 illustrates the Activity Diagram of Teacher, as shown below.

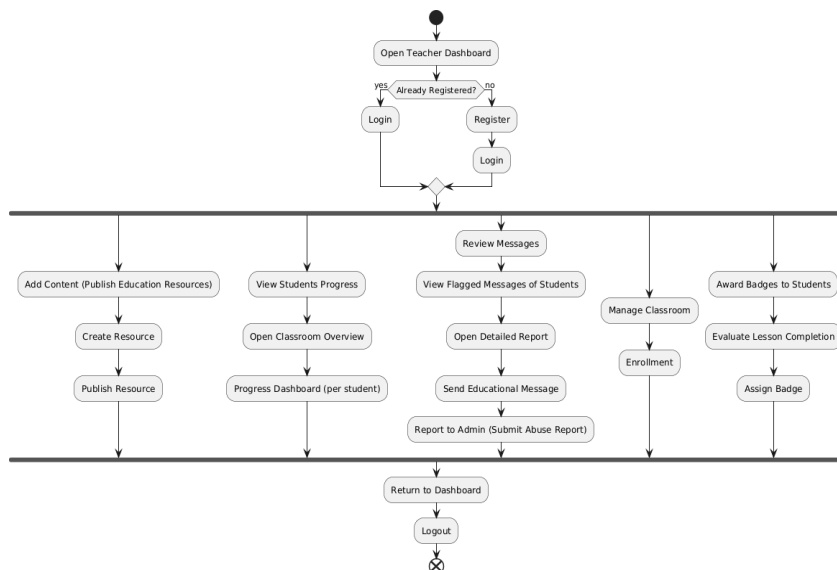


Figure 4. 6:Teacher Activity Diagram

4.3.1 Login– Secure User Authentication

The login flow begins with the Child, Parent, or Teacher providing credentials, which the system validates.

If successful: The respective dashboard (Child, Parent, Teacher, or Admin) is opened.

If failed: An error message is displayed.

The sign-up flow includes providing details, verifying them, and creating an account with role assignment.

4.3.2 Chat & Messaging – Safe Communication

After login, the Child can send or receive messages through the chat module.

- Every message is analysed by the AI Safety Filter before delivery.
- Harmful/toxic messages are blocked, masked, or flagged for review.

4.3.3 AI Safety Monitoring – Detecting Cyberbullying

The system continuously monitors chat interactions.

- If bullying or harmful content is detected → Messages are flagged.
- Flagged messages trigger notifications to Parents, Teachers, and Admins.

4.3.4 Alerts & Reports – Parent/Teacher Awareness

Parents and Teachers receive automated alerts about bullying incidents or suspicious activity.

- The system generates reports with message details, timestamps, and severity level.
- Reports are available for viewing and tracking in dashboards.

4.3.5 Feedback & Complaints – Continuous Improvement

Parents and Teachers can submit feedback on system performance or lodge complaints regarding misuse Admins review complaints and feedback to improve detection accuracy.

4.3.6 Subscription & Payment (For Parents)

Parents can manage subscriptions to unlock premium features like advanced monitoring and extended reporting. The Payment System processes subscription payments securely.

4.3.7 Flow Highlights

The activity diagram separates different modules (User Authentication, Chat & Messaging, AI Safety Monitoring, Alerts & Reports, Feedback & Complaints, Payment & Subscription).

- Decision points (e.g., login success/failure, message safe/unsafe, complaint resolved/unresolved) guide process flow.
- Continuous cycles of monitoring, alerts, and reporting ensure constant protection for children.
- Reward and feedback mechanisms (such as positive notifications for safe interactions) improve trust and engagement among children, parents, and teachers.

4.4 Design Models

The Following Figure 4.7 illustrates the Class diagram of the KidSafe System, as shown below.

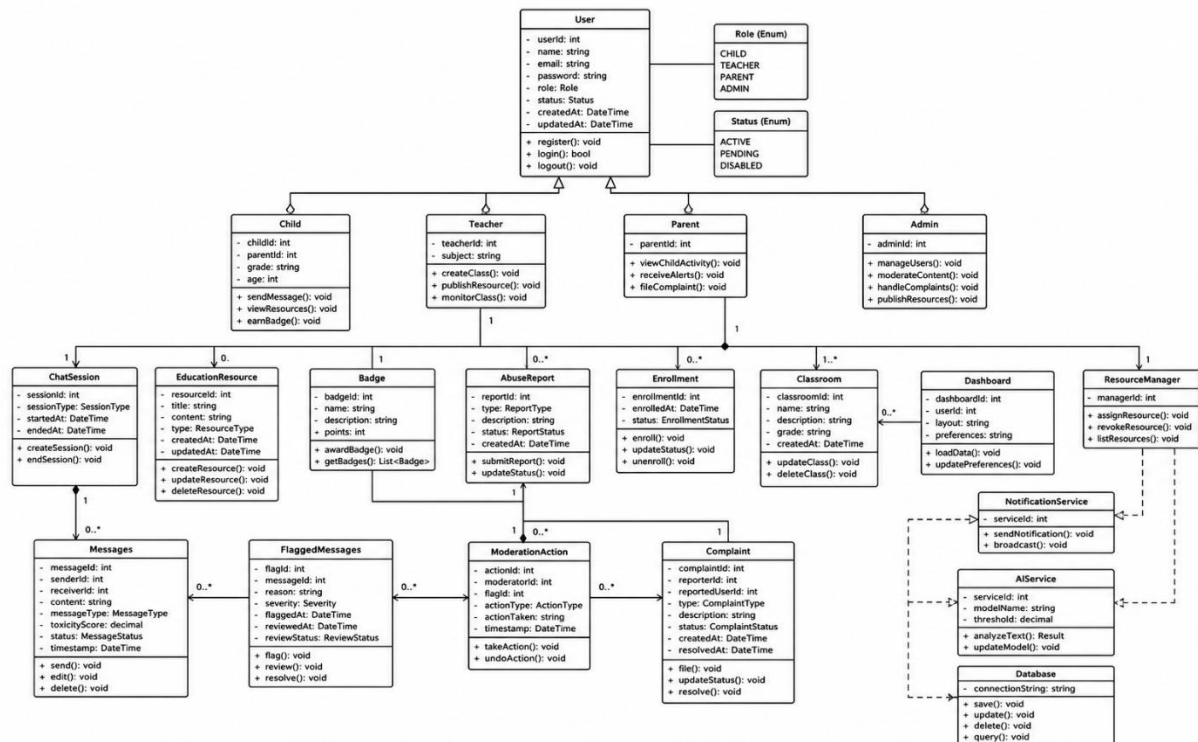


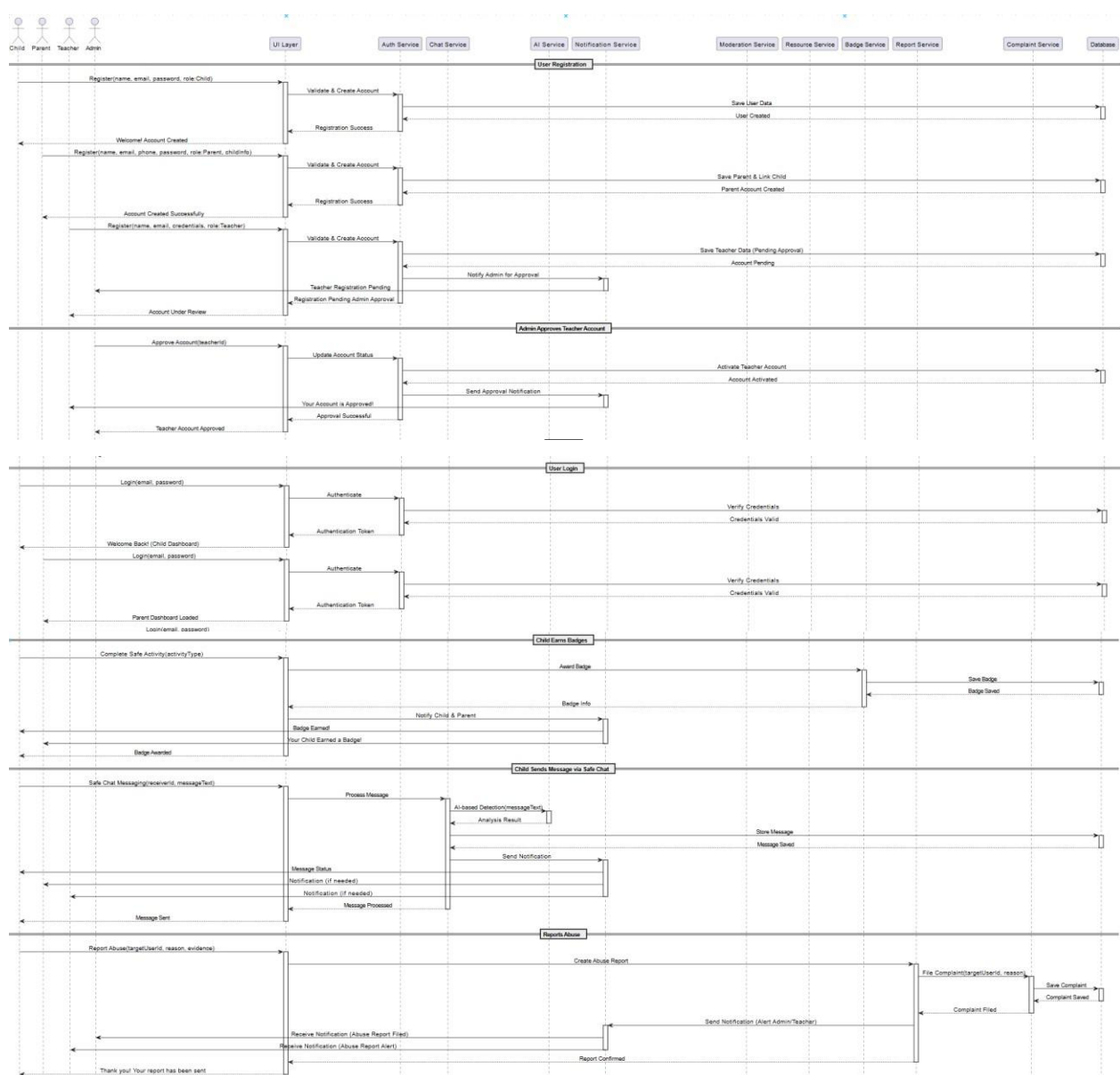
Figure 4. 7: Class Diagram

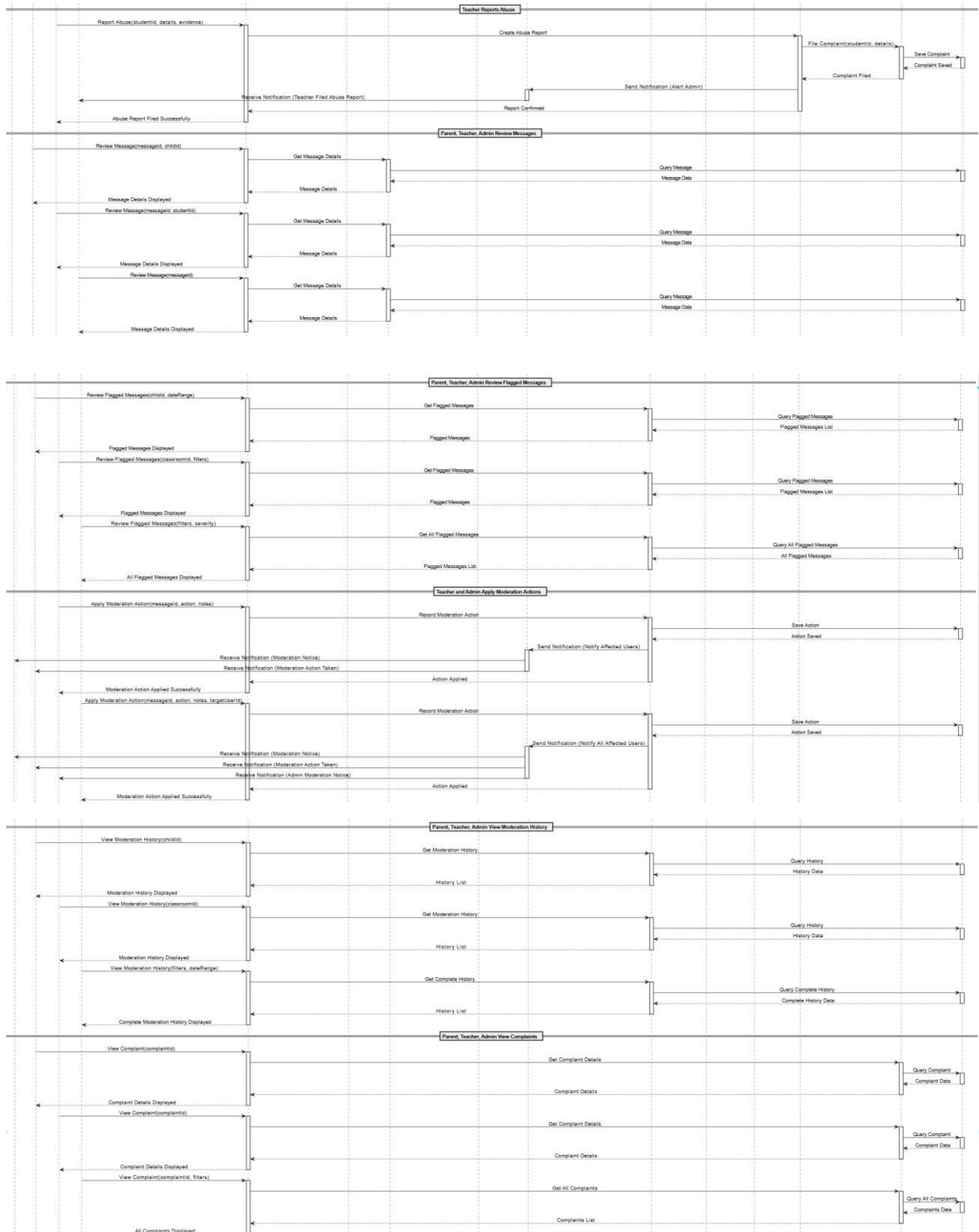
4.4.1 Class Diagram Description

The class diagram of the KidSafe – AI-Based Cyberbullying Detection System illustrates the static structure of the application by defining the major classes, their attributes, methods, and relationships. It consists of nineteen interconnected classes that collectively support secure communication, AI-based cyberbullying detection, parental monitoring, classroom management, complaint handling, and reward-based learning. The User class serves as the base class and is inherited by the Admin, Child, Parent, and Teacher classes, each performing role-specific functions through the Dashboard. The Admin manages users, moderation, complaints, and educational resources, while the Child communicates safely, reports abuse, and accesses learning materials and badges. The Parent monitors the child's online activities, reviews flagged messages, and receives alerts, whereas the Teacher supervises classroom interactions, manages classrooms, and publishes educational resources.

The communication process is handled through the ChatSession and Message classes, where every message is analysed by the AIService before delivery. Harmful messages are stored in the FlaggedMessage class and processed through the ModerationAction class. The AbuseReport and Complaint classes support reporting and complaint management, while the EducationResource, Badge, Enrollment, and Classroom classes facilitate learning, gamification, and classroom administration. The Dashboard provides role-based access to system information, and the NotificationService delivers alerts and announcements to users. Finally, the AIService performs cyberbullying detection using Artificial Intelligence and Natural Language Processing techniques, while the Database securely stores and manages all system data, ensuring efficient communication and overall system functionality.

The Following Figure 4.8 illustrates the Sequence diagram, as shown below.





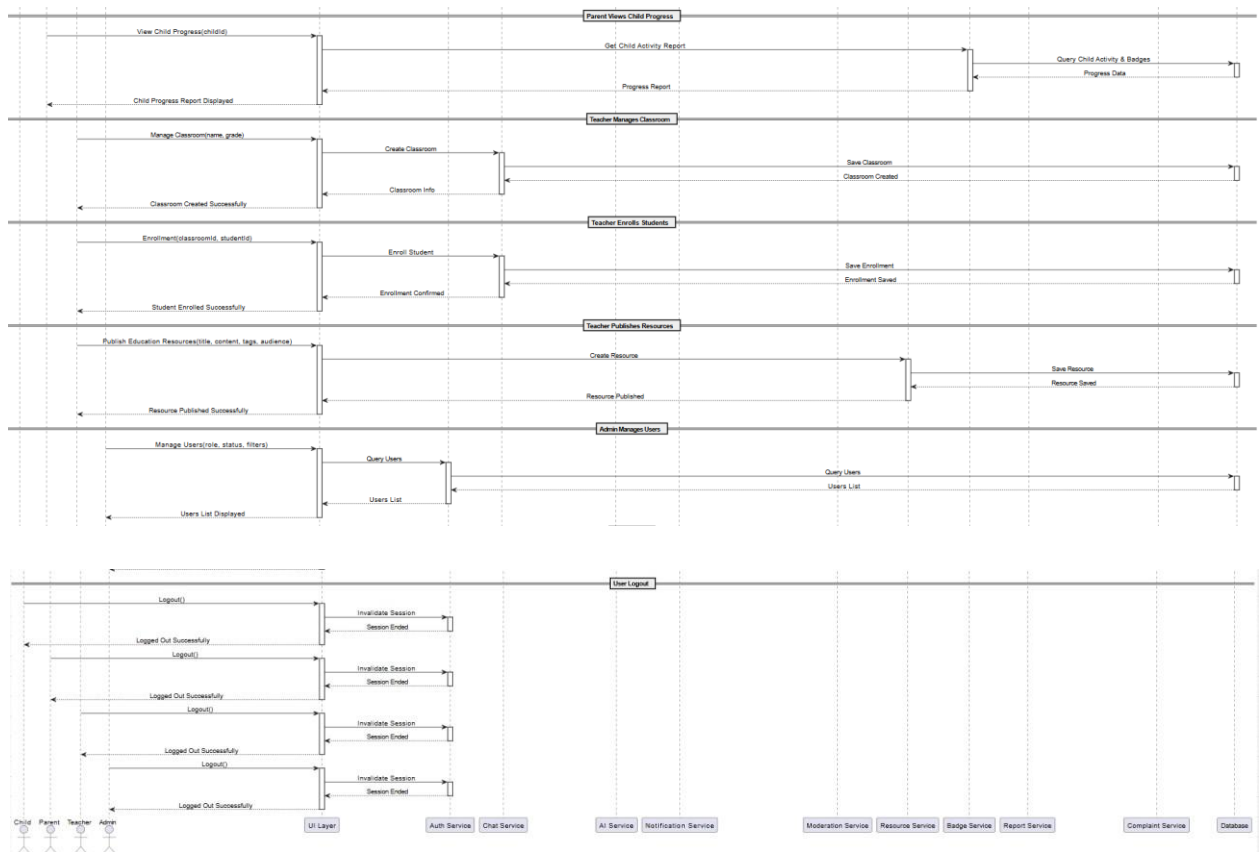


Figure 4. 8: Sequence Diagram

4.4.2 Sequence Diagram Description

The KidSafe – AI-Based Cyberbullying Detection System involves four primary actors: Child, Parent, Teacher, and Admin, each performing role-specific functions within the application. The Child uses the platform to communicate safely, access educational resources, and earn badges, while all chat messages are analysed by the AI Safety Filter before delivery. The Parent monitors the child's activities through the monitoring dashboard, receives alerts, reviews flagged messages and moderation history, and submits feedback or complaints when required. The Teacher manages classrooms, educational resources, student enrollment, and reviews flagged incidents, whereas the Admin oversees overall system operations, manages users, reviews reports and complaints, applies moderation actions, and governs AI safety policies.

The system consists of several interconnected components that coordinate application workflows. The ChatModule manages messaging, classroom communication, and message retrieval, while the AISafetyFilter analyses messages in real time to detect cyberbullying, threats, and inappropriate content before delivery. The MonitoringDashboard provides role-based access for Parents, Teachers, and Admins to review flagged content, moderation history, and user activities. The AlertSystem delivers real-time notifications to all users regarding approvals, harmful activities, and

important system events, while the ReportSystem records flagged incidents and integrates with the ComplaintService to manage formal complaints.

Additional components enhance the overall functionality of the system. The ModerationModule records and tracks moderation actions, providing a complete history for review and auditing. The ResourceService enables teachers to publish educational materials for children, while the BadgeService rewards positive online behaviour and allows parents to monitor their child's progress. The ComplaintService manages complaint submission and tracking by routing cases to the appropriate authorities, and the FeedbackSystem collects feedback and suggestions from Children, Parents, and Teachers, allowing the Admin to review and address them for continuous system improvement.

4.4.3 System Diagram

The KidSafe – AI-Based Cyberbullying Detection System follows a three-layer architecture that ensures the application is modular, scalable, and secure.

The User Interface Layer serves as the interaction point between users and the system by providing dedicated dashboards for Children, Parents, Teachers, and Admins according to their roles. It also includes a real-time Chat Window that enables safe communication through AI moderation and an Alerts Panel that displays notifications for flagged messages and other important system events. This layer provides users with an intuitive interface to communicate, monitor activities, manage classrooms, and administer the overall system.

The Application Logic Layer contains the core business logic and controls the primary functionalities of the system. It manages user registration, authentication, and role-based access, while supporting classroom management, student enrollment, and real-time messaging. The integrated AI Safety Filter analyses messages to detect cyberbullying, threats, toxicity, and inappropriate content before delivery, automatically triggering alerts when harmful behaviour is identified. This layer also includes the Report System for incident logging and moderation tracking, the Alerts Module for sending real-time notifications, and the Consultation and Feedback component, which enables parents and teachers to submit feedback, appeals, or concerns.

The Data Storage Layer provides secure and structured storage for all application data. It maintains user profiles, authentication credentials, chat messages, moderation results, flagged content, incident reports, analytics, feedback, complaints, consultation records, and educational resources. By ensuring reliable storage and efficient data retrieval, this layer supports the continuous operation, security, and overall performance of the KidSafe system while enabling seamless communication between all system components.

The Following Figure 4.9 illustrates the system diagram of child, as shown below.

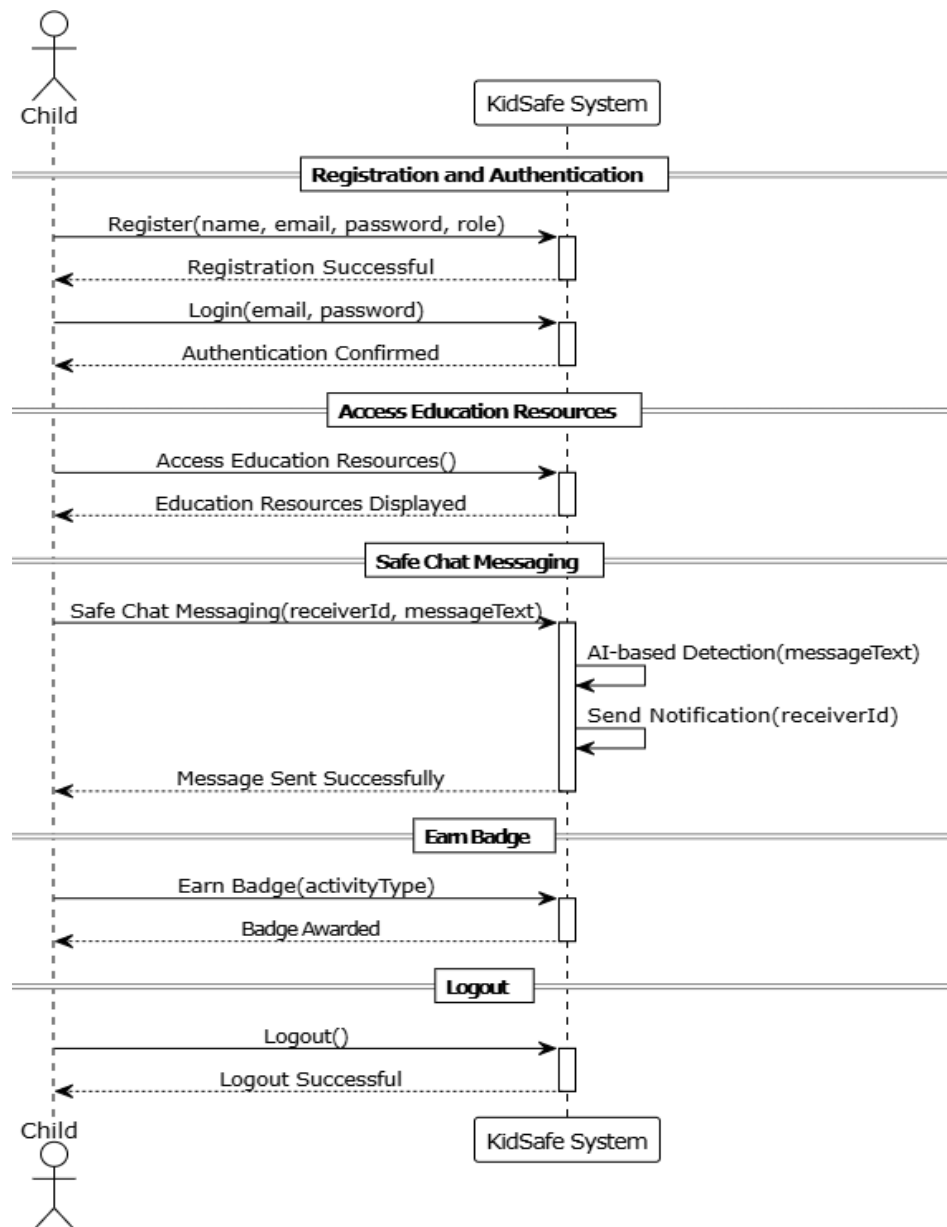


Figure 4. 9: Child System Diagram

The Following Figure 4.10 illustrates the system diagram of admin, as shown below.

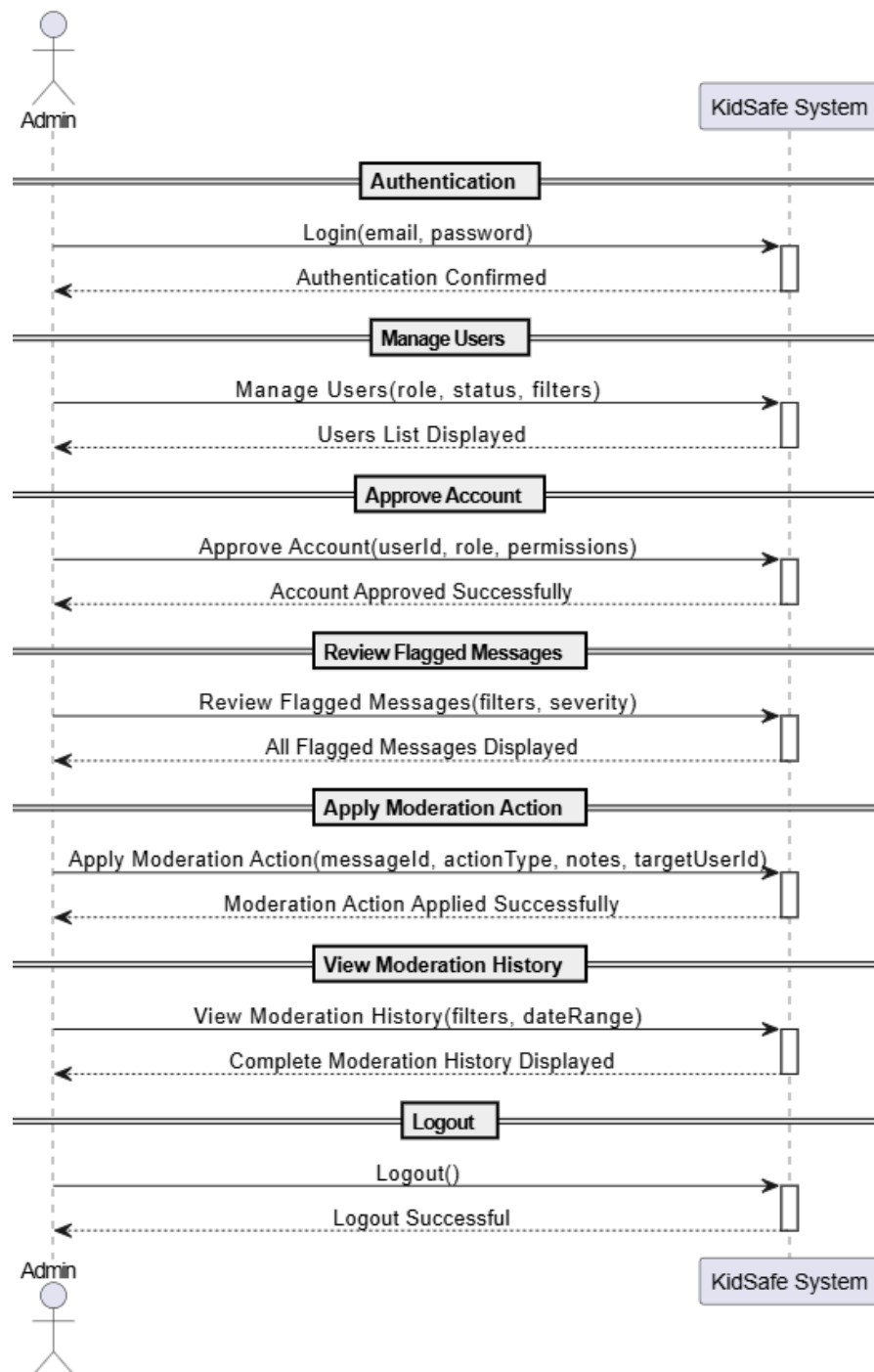


Figure 4. 10: Admin System Diagram

The Following Figure 4.11 illustrates the system diagram of parent, as shown below.

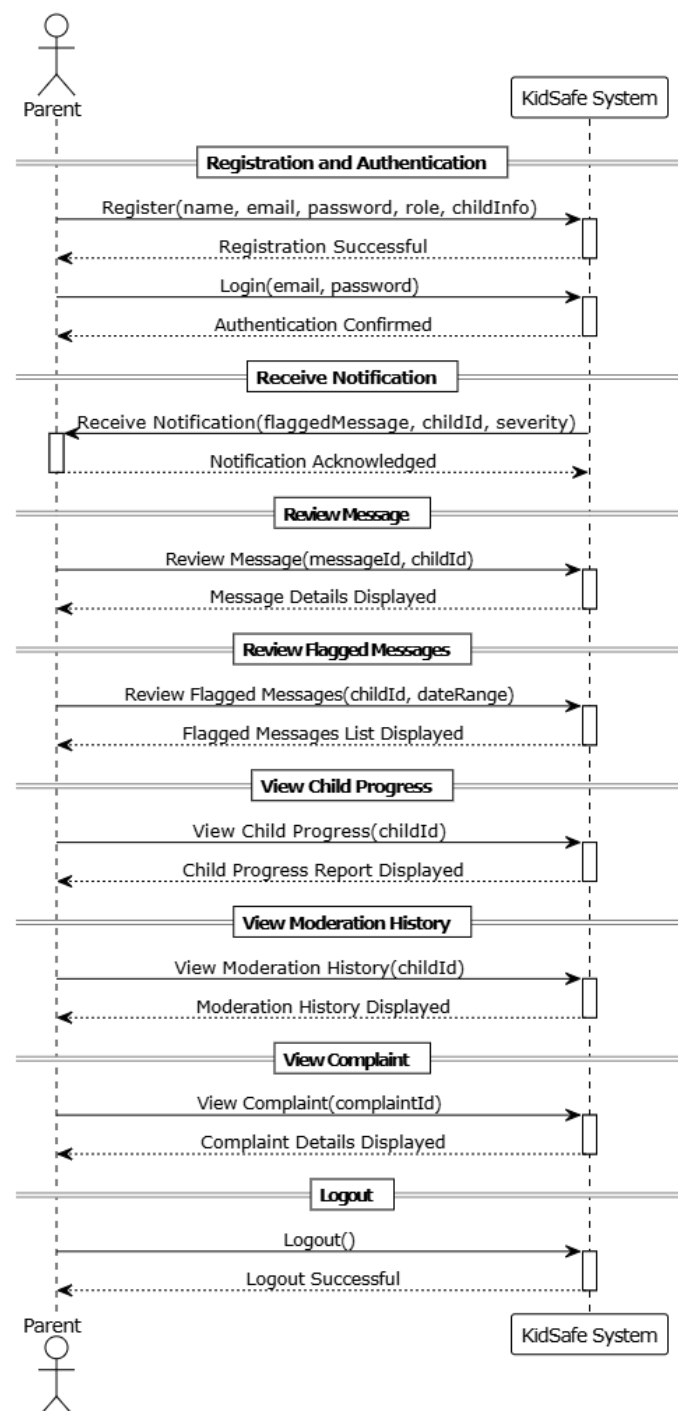


Figure 4. 11:Parent System Diagram

The Following Figure 4.12 illustrates the system diagram of teacher, as shown below.

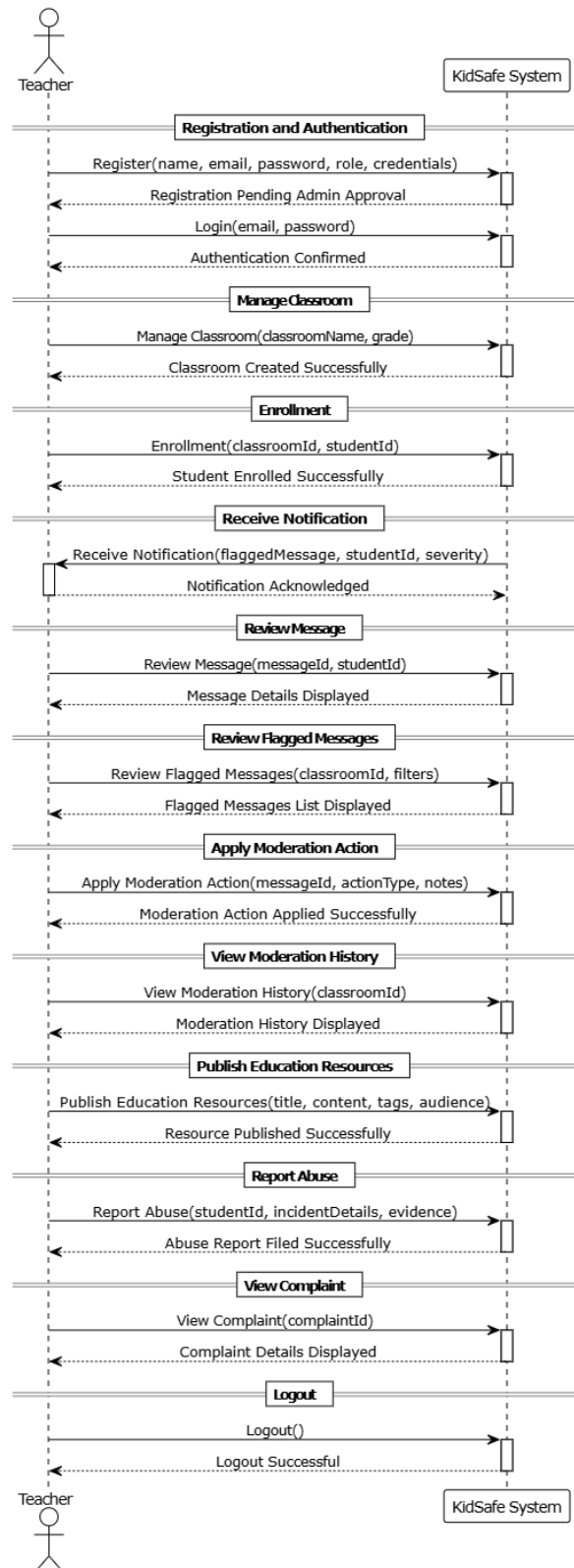


Figure 4. 12: Teacher System Diagram

5. Implementation

This chapter presents the implementation details of the proposed KidSafe – AI-Based Cyberbullying Detection App. The system has been implemented using a multi-tier architecture comprising a Blazor WebAssembly frontend, ASP.NET Core Web API backend, and a Python FastAPI AI microservice. The implementation integrates Artificial Intelligence (AI), real-time communication, secure authentication, cloud notifications, and a reward-based mechanism to create a safe online chatting environment for children. The implementation is supported by various UML diagrams presented in previous chapters.

5.1 Algorithm

The main functionality of KidSafe revolves around detecting cyberbullying before a message reaches the intended recipient. Whenever a child sends a message, it is first analyzed by the AI service. Based on the toxicity score returned by the AI model, the message is classified as Safe, Flagged, or Blocked. Safe messages are delivered normally and reward the child with points. Flagged messages are masked before delivery and generate alerts for parents and teachers, whereas blocked messages are completely prevented from being delivered.

5.1.1 AI-Based Message Moderation

```
Start
Receive message from child user.
Send the message to the AI Microservice.
Receive toxicity score from AI model.
IF score < 0.5 THEN
    Label = Safe
    Deliver message to receiver
    Award 10 reward points
    Return "Sent"
ELSE IF score >= 0.5 AND score < 0.8 THEN
    Label = Flagged
    Mask offensive words
    Store message in FlaggedMessages table
```

```
    Notify parents and teachers
    Deliver masked message
    Return "Masked"
ELSE
    Label = Blocked
    Store message in FlaggedMessages table
    Notify parents and teachers
    Block message delivery
    Return "Blocked"
End
```

5.1.2 User Authentication

```
Start
User enters Email and Password
Validate Input
IF Credentials are Correct THEN
    Authenticate User
    Identify User Role
    Generate JWT Token
    Redirect to Respective Dashboard
ELSE
    Display "Invalid Credentials"
ENDIF
Stop
```

5.1.3 AI-Based Cyberbullying Detection

```
Start
User Types Message
Send Message to AI Detection Service
Preprocess Message
Predict Message Label
```

```
IF Label = Safe THEN
    Deliver Message
ELSE IF Label = Offensive THEN
    Flag Message
    Store Incident
    Notify Parent and Teacher
ELSE IF Label = Severe Toxic THEN
    Block Message
    Store Incident
    Send Immediate Alert
ENDIF
Save Detection Result
Stop
```

5.1.4 Safe Chat Process

```
Start
Authenticated User Sends Message
Receive Message
Forward to AI Safety Filter
IF Message is Safe THEN
    Deliver to Receiver
ELSE
    Apply Moderation
    Save Flagged Message
ENDIF
Update Chat History
Stop
```


5.1.5 Parent/Teacher Alert Generation

```
Start
Receive AI Detection Result
IF Message is Flagged OR Blocked THEN
    Create Alert
    Retrieve Parent Details
    Retrieve Teacher Details
    Send Notification
    Log Alert History
ENDIF
Stop
```

5.1.6 Abuse Reporting Process

```
Start
Child Clicks "Report Abuse"
Enter Report Details
Save Report
Notify Teacher and Parent
IF Complaint Required THEN
    Forward to Admin
ENDIF
Update Report Status
Stop
```

5.2 External APIs

The KidSafe application integrates several external APIs and frameworks to implement artificial intelligence, real-time communication, authentication, notifications, and database operations. These APIs simplify the development process while improving the reliability, security, and scalability of the application.

The Following Table 5.1 Shows the Details of APIs, as shown below

Table 5. 1:Details of APIs Used in the Project

Name of API / Framework	Description of API	Purpose of Usage	Function / Class Name in which it is Used
FastAPI	A high-performance Python web framework used to develop RESTful AI services.	Provides the AI microservice that analyzes chat messages and returns toxicity labels and scores.	main.py, AIService.AnalyzeAsync(), classifier.predict()
SignalR	ASP.NET Core framework for real-time client-server communication.	Enables instant chat messaging, typing indicators, and real-time dashboard alerts.	ChatHub.cs, ChatHubService.cs, ReceiveMessage(), FlaggedMessageAlert()
Firebase Cloud Messaging (FCM)	Google's cloud messaging service for push notifications.	Sends instant notifications to parents and teachers whenever a flagged or blocked message is detected.	NotificationService.cs, BroadcastToParentsAsync(), RegisterFcmTokenAsync()
JWT Bearer Authentication	Token-based authentication mechanism for securing web APIs.	Authenticates users and authorizes access to protected API endpoints.	AuthService.cs, GenerateJwt(), AuthController.cs

Entity Framework Core (EF Core)	Object-Relational Mapping (ORM) framework for .NET applications.	Performs CRUD operations and manages communication with the SQLite database.	AppDbContext.cs, Users, FlaggedMessages, Rewards
BCrypt.Net	Password hashing library for .NET applications.	Encrypts user passwords before storing them securely in the database.	AuthService.cs, HashPassword(), VerifyPassword()

5.3 User Interface

The KidSafe – AI-Based Cyberbullying Detection System provides a modern, responsive, and user-friendly interface designed to ensure an intuitive experience for children, parents, teachers, and administrators. The user interface was developed using Blazor WebAssembly, following a clean and consistent design approach that incorporates gradient color themes, rounded UI components, responsive layouts, smooth animations, and intuitive navigation. Each interface is carefully designed according to the responsibilities and permissions of its respective user role, allowing users to perform their tasks efficiently without unnecessary complexity. Child users are provided with a simple and engaging interface for secure messaging, educational resources, and badge tracking, while parents have access to monitoring dashboards, alerts, and reports related to their child's online activities. Teachers can manage classrooms, review flagged messages, publish educational resources, and monitor student interactions, whereas administrators are provided with comprehensive system management features, including user administration, moderation controls, complaint handling, and AI policy management. The interface also emphasizes usability by ensuring clear navigation, readable typography, consistent icons, and well-organized content, enabling users to access information quickly. Furthermore, responsive design techniques ensure that the application functions effectively across different screen sizes and devices, providing a seamless and accessible user experience while supporting the overall objective of creating a safe and secure digital communication environment for children.

The Following Figure 5.1 shows the User Interface of Splash Screen



Figure 5. 1: Splash Screen

The Following Figure 5.2 shows the User Interface of Login Screen

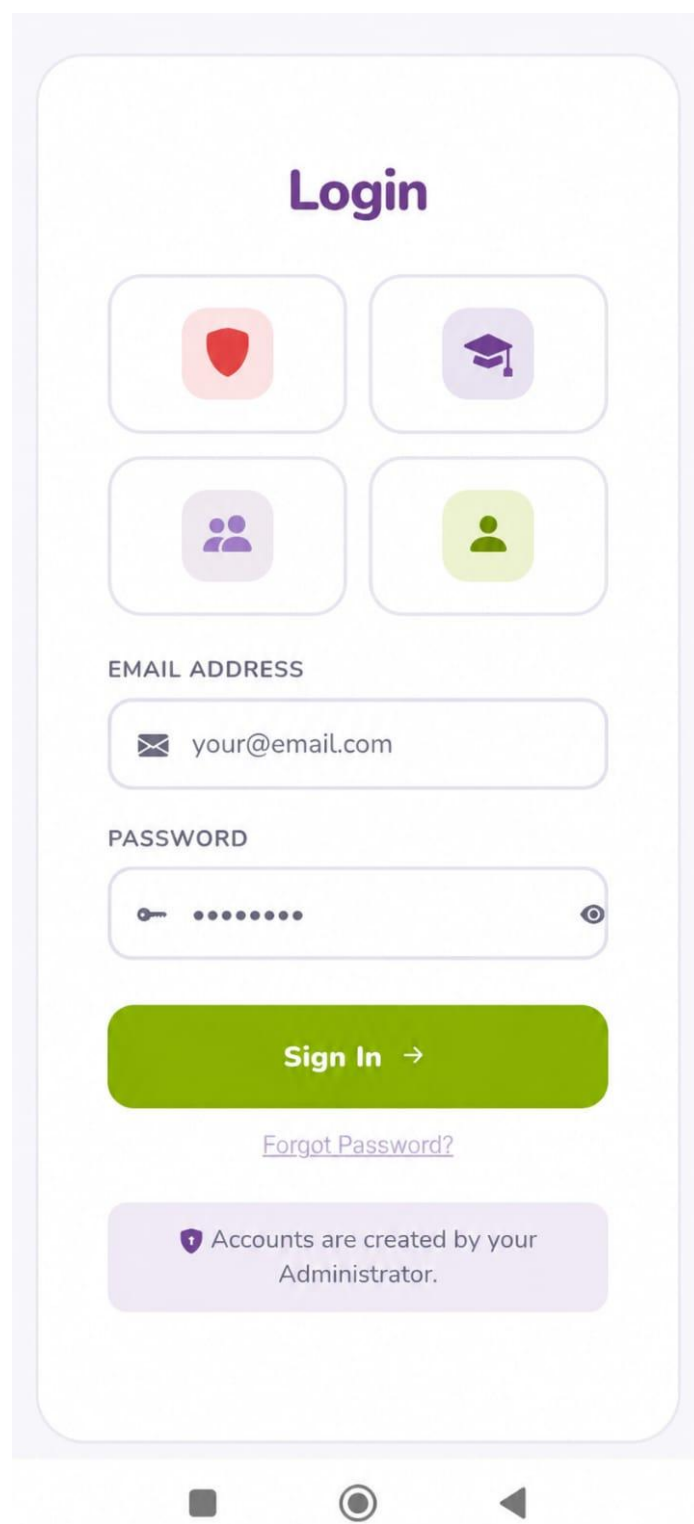


Figure 5. 2:Login Screen

The Following Figure 5.3 shows the User Interface of My Profile Screen

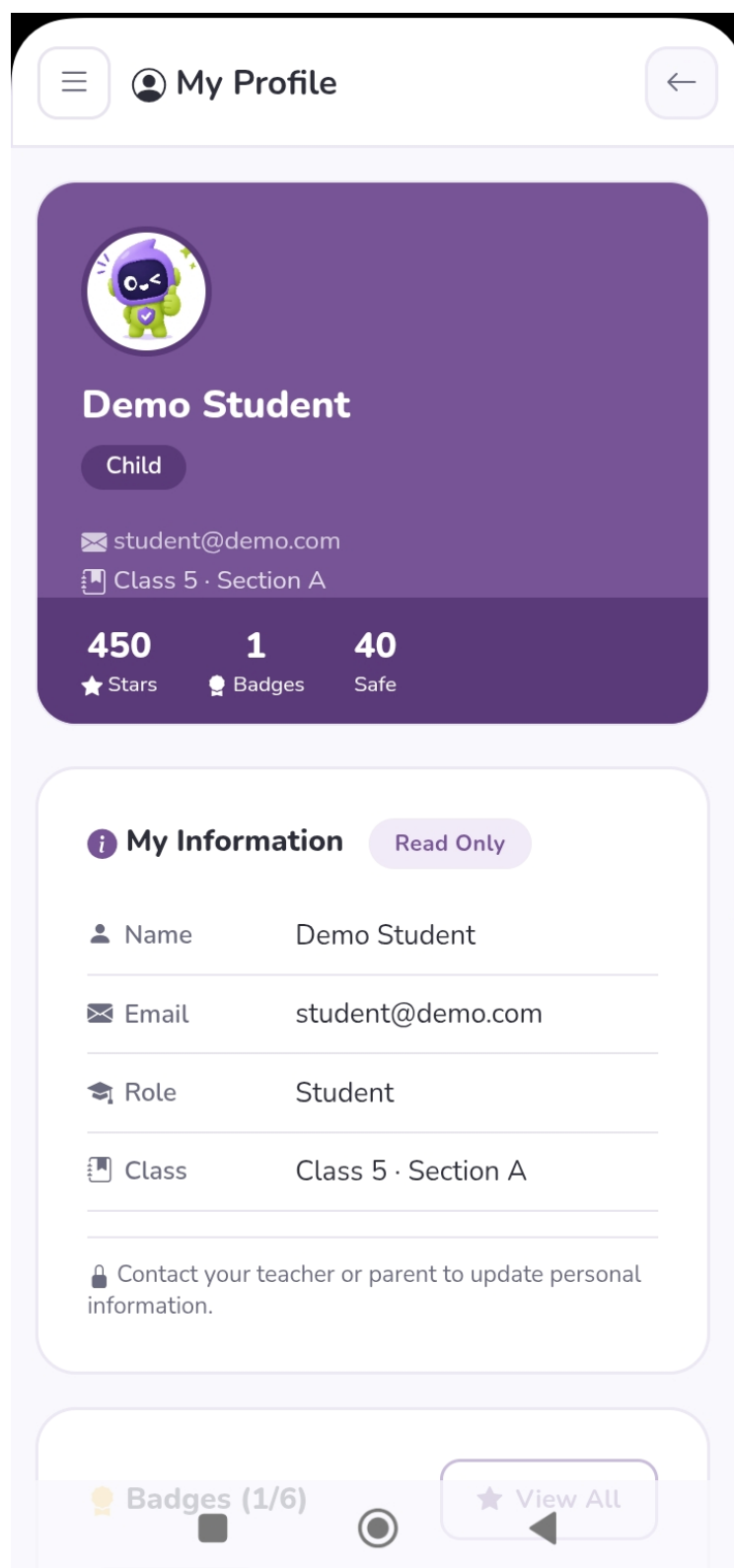


Figure 5. 3: My Profile

The Following Figure 5.4 shows the User Interface of Child Chat Screen

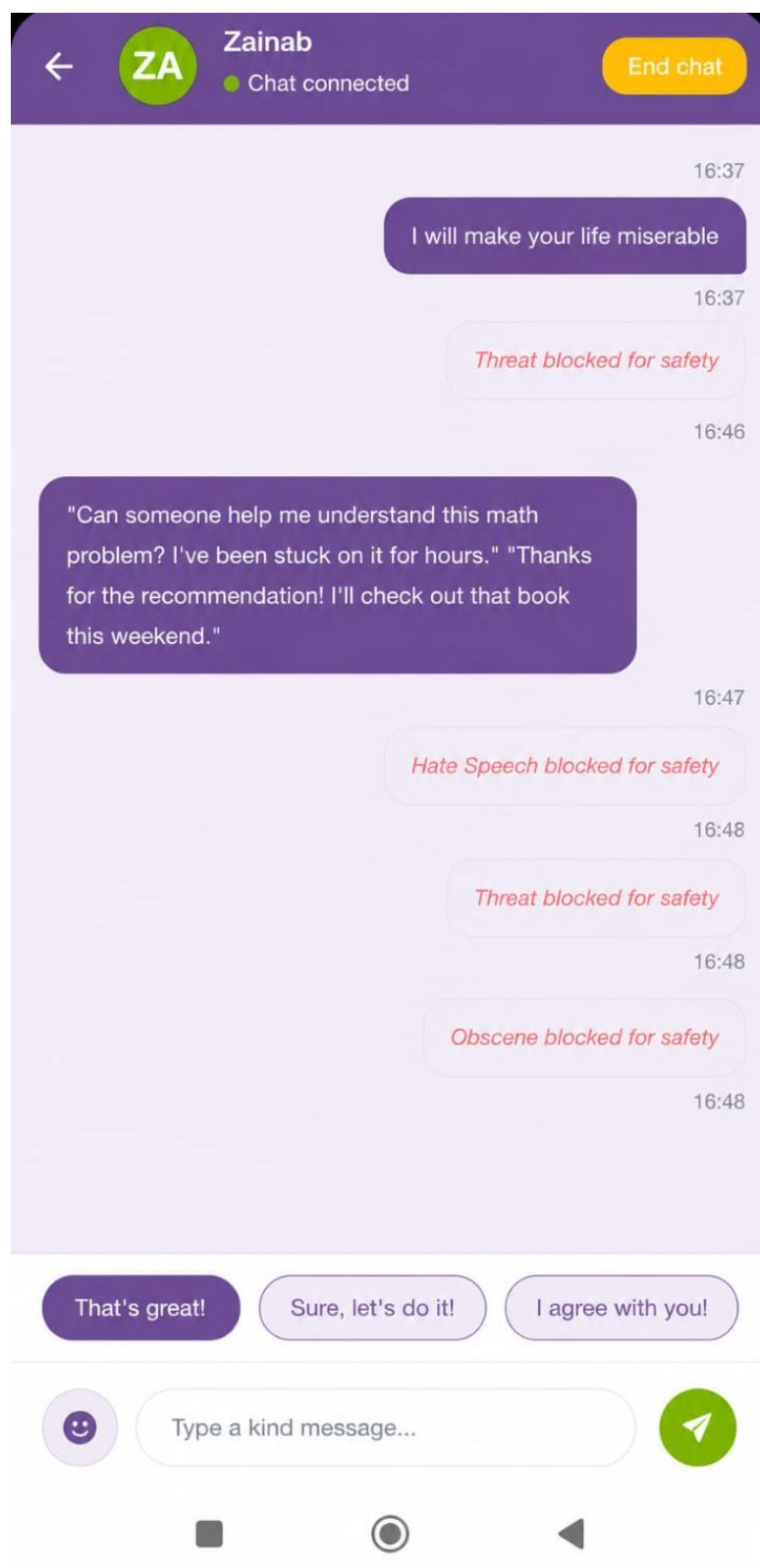


Figure 5. 4: Child Chat Interface

The Following Figure 5.5 shows the User Interface of Rewards Screen

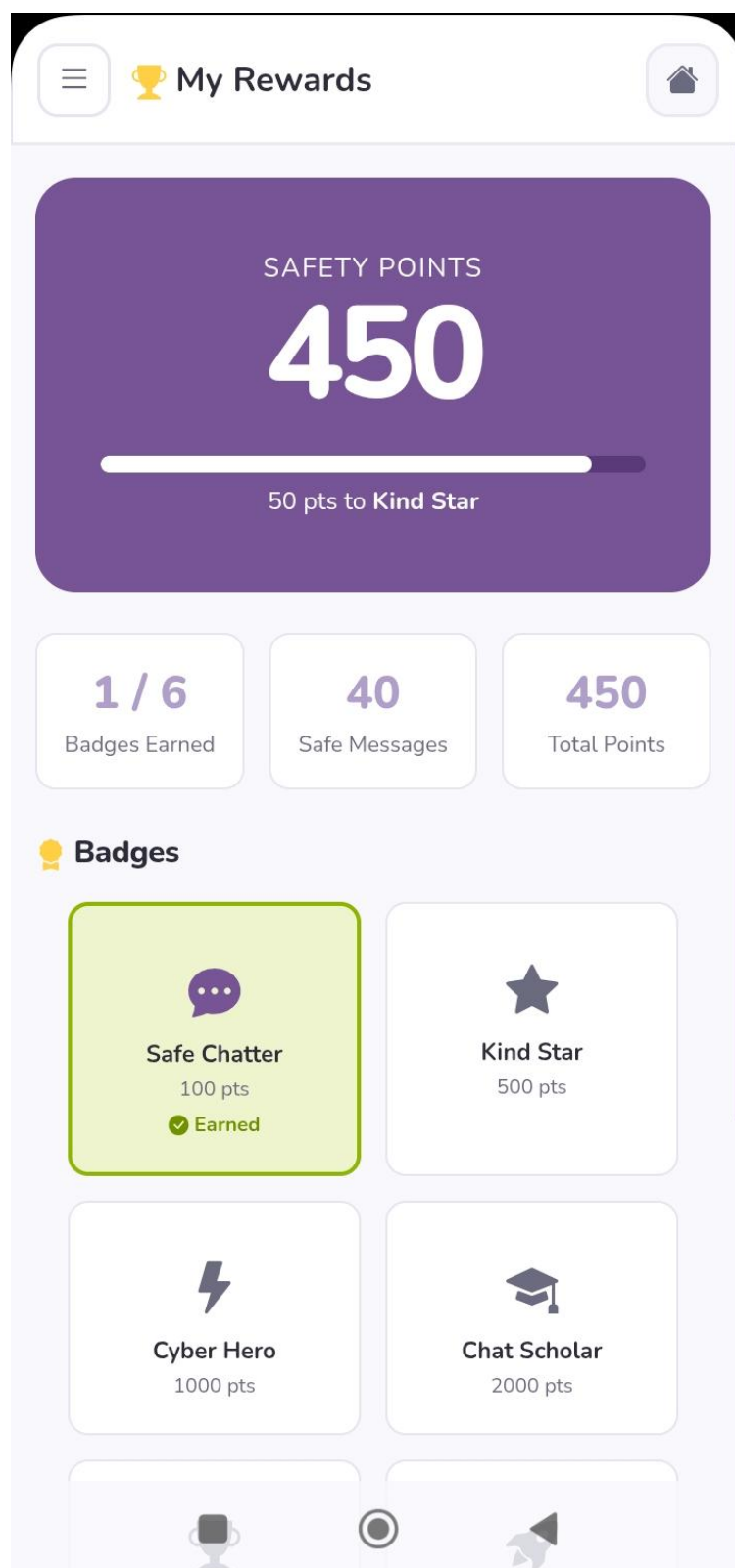


Figure 5. 5: Rewards Page

The Following Figure 5.6 shows the User Interface of Parent Dashboard Screen

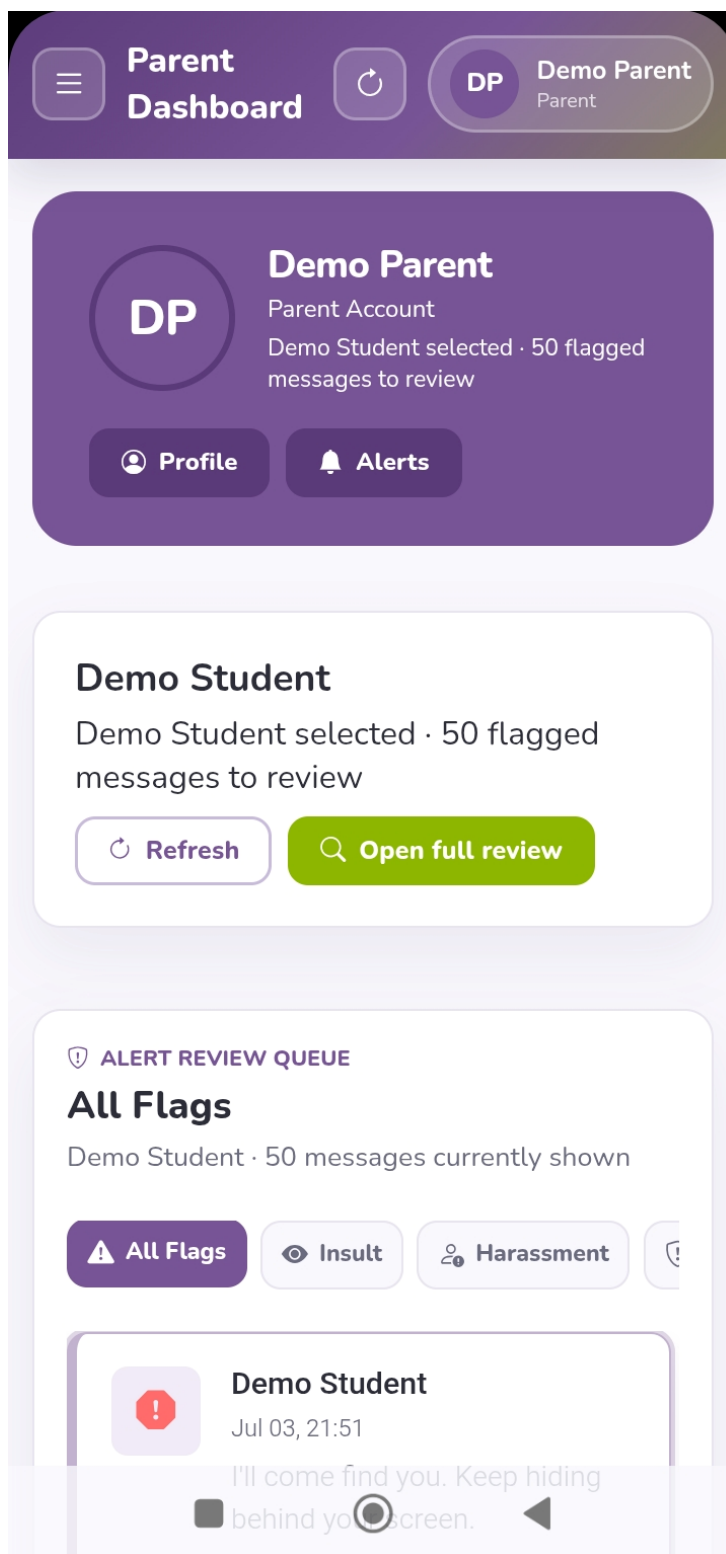


Figure 5. 6:Parent Dashboard

The Following Figure 5.7 shows the User Interface of Teacher Panel Screen



Figure 5. 7:Teacher Panel

The Following Figure 5.8 shows the User Interface of Admin Panel Screen

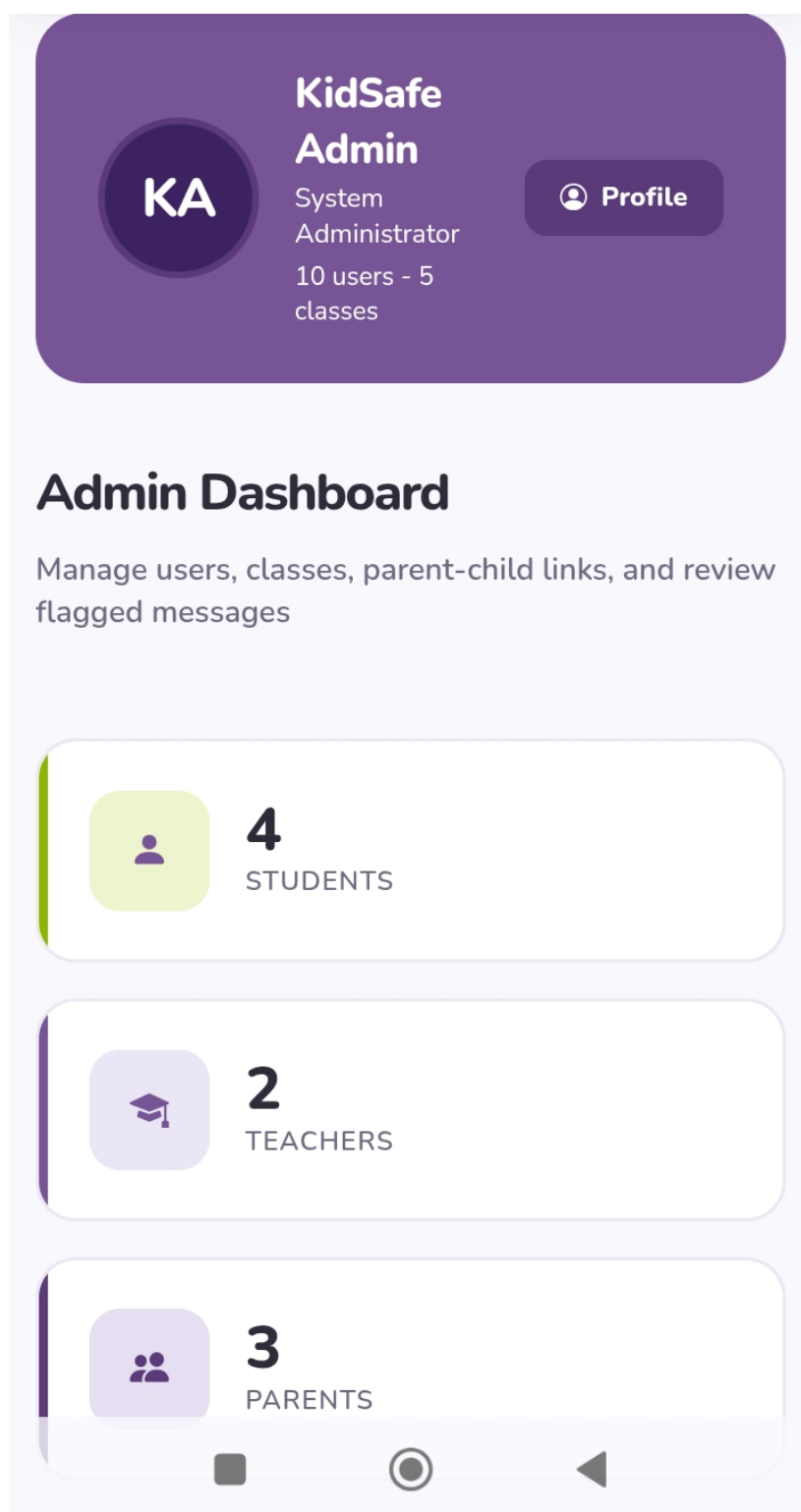


Figure 5. 8:Admin Panel

The Following Figure 5.9 shows the User Interface of Moderation Action Screen

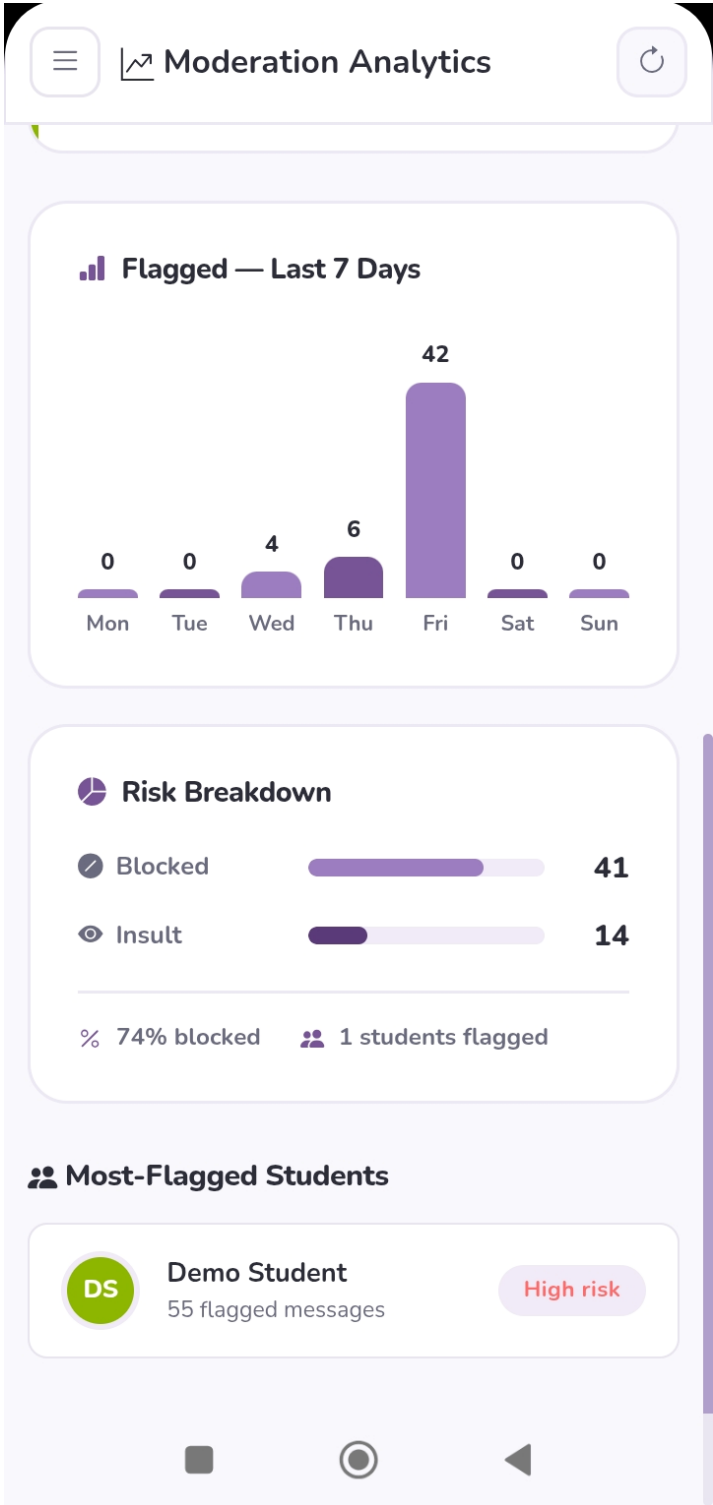


Figure 5. 9: Moderation Action Screen

6. Testing and Evaluation

Testing is one of the most important phases in the software development life cycle as it ensures that the developed system satisfies all functional and non-functional requirements. The purpose of testing is to identify defects, verify system functionality, validate user requirements, and ensure the reliability of the proposed system before deployment.

For the KidSafe – AI-Based Cyberbullying Detection App, both manual testing and automated testing were performed. Manual testing was conducted to verify the correctness of each module, whereas automated testing was used to validate API endpoints and the AI microservice. The testing process covered authentication, AI message moderation, real-time communication, notification delivery, reward management, and dashboard functionalities.

6.1 Manual Testing

Manual testing was carried out by executing different test scenarios without using automated testing tools. Each module of the KidSafe application was tested individually and then integrated with other modules to ensure the entire system functioned correctly. The manual testing process included System Testing, Unit Testing, Functional Testing, and Integration Testing.

6.1.1 System Testing

System testing was performed after the successful implementation of the KidSafe application. During this phase, the complete system was tested as a whole to verify that all modules interacted correctly and fulfilled the project requirements.

The following components were tested during system testing:

- User Registration
- User Login
- Child Chat Module
- AI-Based Message Moderation
- Reward System
- Parent Dashboard
- Teacher Panel
- Firebase Push Notifications
- Real-Time Messaging using SignalR
- Database Operations

The testing confirmed that all modules worked together successfully. Messages were correctly classified as Safe, Flagged, or Blocked, reward points were

awarded for safe communication, parents and teachers received notifications for inappropriate messages, and all data were stored correctly in the SQLite database.

6.1.2 Unit Testing

Unit testing was performed to verify the correctness of individual modules of the KidSafe application. Each module was tested independently to ensure that it performed its intended functionality before integrating it with other modules.

- **Unit Testing 1: User Login**

Testing Objective: To ensure that registered users can successfully log in using valid credentials.

The Following Table 6.1 illustrates the Login unit test case as shown below

Table 6. 1:Login Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Login using valid Child account	Email: <u>child@test.com</u> Password: *****	Child Dashboard opens successfully	Pass
2	Login using valid Parent account	Email: <u>parent@test.com</u> Password: *****	Parent Dashboard opens successfully	Pass
3	Login using valid Teacher account	Email: <u>teacher@test.com</u> Password: *****	Teacher Panel opens successfully	Pass
4	Login using invalid password	Valid Email, Wrong Password	Error message displayed	Pass
5	Login using unregistered email	Invalid Email	Login denied	Pass

- **Unit Testing 2: User Registration**

Testing Objective: To ensure that new users can successfully create an account.

The Following Table 6.2 illustrates the Registration unit test case as shown below

Table 6. 2:Registration Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Register Child account	Valid information	Account created successfully	Pass
2	Register Parent account	Valid information	Account created successfully	Pass
3	Register Teacher account	Valid information	Account created successfully	Pass
4	Register with duplicate email	Existing Email	Registration rejected	Pass
5	Register with empty required fields	Empty inputs	Validation message displayed	Pass

- **Unit Testing 3: AI Message Moderation**

Testing Objective: To verify that the AI model correctly classifies chat messages based on toxicity.

The Following Table 6.3 illustrates the Moderation unit test case as shown below

Table 6. 3:AI Moderation Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Send positive message	"Hello! How are you?"	Classified as Safe	Pass
2	Send mildly offensive message	Offensive words	Classified as Flagged	Pass
3	Send highly abusive message	Severe abusive content	Classified as Blocked	Pass
4	Verify toxicity score generation	Sample message	Score returned between 0.0–1.0	Pass

- **Unit Testing 4: Reward System**

Testing Objective: To ensure reward points are awarded only for safe messages.

The Following Table 6.4 illustrates the Reward System unit test case as shown below

Table 6. 4:Reward System Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Send Safe message	Score < 0.5	10 points awarded	Pass
2	Send Flagged message	Score 0.5–0.8	No points awarded	Pass
3	Send Blocked message	Score $\geq$ 0.8	No points awarded	Pass
4	Reach badge threshold	100 Points	Badge unlocked	Pass

- **Unit Testing 5: Parent Dashboard**

Testing Objective: To ensure the Parent Dashboard correctly displays cyberbullying reports.

The Following Table 6.5 illustrates the Parent Dashboard unit test case as shown below

Table 6. 5:Parent Dashboard Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	View flagged messages	Parent Login	Flagged message list displayed	Pass
2	View dashboard statistics	Parent Login	Statistics loaded successfully	Pass
3	Receive live notification	Flagged message generated	Notification displayed instantly	Pass

- **Unit Testing 6: Teacher Panel**

Testing Objective: To verify that teachers can monitor classroom cyberbullying activities.

The Following Table 6.6 illustrates the Teacher Panel unit test case as shown below

Table 6. 6:Teacher Panel Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	View classroom statistics	Teacher Login	Statistics displayed correctly	Pass
2	View incident reports	Teacher Login	Incident table loaded	Pass
3	View risk indicators	Teacher Login	Risk indicators displayed correctly	Pass

- **Unit Testing 7: Firebase Push Notifications**

Testing Objective: To ensure push notifications are delivered to parents and teachers when harmful messages are detected.

The Following Table 6.7 illustrates the Notification unit test case as shown below

Table 6. 7:Firebase Notification Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Send Flagged message	Parent subscribed to FCM	Push notification received	Pass
2	Send Blocked message	Parent subscribed to FCM	Push notification received	Pass
3	Safe message	Score < 0.5	No notification sent	Pass

- **Unit Testing 8: SignalR Real-Time Messaging**

Testing Objective: To verify that chat messages are delivered instantly through SignalR.

The Following Table 6.8 illustrates the SignalR unit test case as shown below

Table 6. 8:SignalR Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Send safe message	Child → Child	Receivers get message instantly	Pass
2	Typing indicator	Sender typing	Receiver sees typing status	Pass
3	Connection loss	Disconnect internet	Reconnection performed automatically	Pass

6.1.3 Functional Testing

Functional testing was performed after the successful completion of unit testing to verify that every module of the KidSafe application functioned according to the specified functional requirements. The purpose of this testing was to ensure that each feature produced the expected output under normal operating conditions. The modules tested include authentication, AI-based message moderation, child chat, reward management, parent dashboard, teacher panel, push notifications, and real-time communication.

- **Functional Testing 1: User Login**

Testing Objective: To verify that users are redirected to the correct interface according to their assigned role after successful authentication.

The Following Table 6.9 illustrates the Login Functional test case as shown below

Table 6. 9:Login Functional Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Login as Child	Valid Child credentials	Child Chat page is loaded successfully	Pass
2	Login as Parent	Valid Parent credentials	Parent Dashboard is loaded successfully	Pass
3	Login as Teacher	Valid Teacher credentials	Teacher Panel is loaded successfully	Pass

- **Functional Testing 2: AI Message Classification**

Testing Objective: To verify that the AI moderation service correctly classifies messages into Safe, Flagged, and Blocked categories.

The Following Table 6.10 illustrates the Message Classification Functional test case as shown below

Table 6. 10: AI Message Classification Functional Test Case

No.	Test Case / Test Script	Input Message	Expected Result	Result
1	Send positive message	"Good morning, friend!"	Message classified as Safe and delivered	Pass
2	Send mildly offensive message	Message containing inappropriate language	Message classified as Flagged and masked	Pass
3	Send abusive message	Highly offensive message	Message classified as Blocked and not delivered	Pass

- **Functional Testing 3: Child Chat Module**

Testing Objective: To verify that children can communicate securely using the chat interface.

The Following Table 6.11 illustrates the Child Chat Functional test case as shown below

Table 6. 11: Child Chat Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	Send Safe message	Child User	Receiver receives original message	Pass
2	Send Flagged message	Child User	Receiver receives masked message	Pass
3	Send Blocked message	Child User	Message is blocked	Pass

4	Send Emoji	Emoji selected	Emoji delivered successfully	Pass
5	Use Quick Reply	Quick reply button	Message sent successfully	Pass

- **Functional Testing 4: Reward System**

Testing Objective: To verify that reward points and badges are managed correctly.

The Following Table 6.12 illustrates the Rewards Functional test case as shown below

Table 6. 12:Reward System Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	Send Safe message	Safe classification	10 reward points added	Pass
2	Reach badge threshold	100 points	Safe Chatter badge unlocked	Pass
3	View Rewards page	Child account	Updated points and badges displayed	Pass

- **Functional Testing 5: Parent Dashboard**

Testing Objective: To verify that parents can monitor cyberbullying incidents effectively.

The Following Table 6.13 illustrates the Parent Dashboard Functional test case as shown below

Table 6. 13:Parent Dashboard Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	View Dashboard	Parent Login	Dashboard loads successfully	Pass
2	View Flagged Messages	Existing flagged records	Flagged messages displayed	Pass

3	View Statistics	Parent Dashboard	Statistics displayed correctly	Pass
4	Receive Live Alert	Flagged message generated	Alert displayed instantly	Pass

- **Functional Testing 6: Teacher Panel**

Testing Objective: To verify that teachers can monitor classroom communication effectively.

The Following Table 6.14 illustrates the Teacher Panel Functional test case as shown below

Table 6. 14:Teacher Panel Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	Open Teacher Panel	Teacher Login	Panel opens successfully	Pass
2	View Classroom Risk Indicators	Existing data	Risk indicators displayed correctly	Pass
3	View Incident Reports	Existing records	Incident table displayed	Pass

- **Functional Testing 7: Firebase Push Notifications**

Testing Objective: To verify that push notifications are sent whenever inappropriate content is detected.

The Following Table 6.15 illustrates the Notification Functional test case as shown below

Table 6. 15:Firebase Notification Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	Generate Flagged message	Parent subscribed	Push notification received	Pass
2	Generate Blocked message	Parent subscribed	Push notification received	Pass
3	Safe message	Safe classification	No notification generated	Pass

- **Functional Testing 8: SignalR Real-Time Communication**

Testing Objective: To verify that real-time communication functions correctly.

The Following Table 6.16 illustrates the SignalR Functional test case as shown below

Table 6. 16:SignalR Functional Test Case

No.	Test Case / Test Script	Attribute	Expected Result	Result
1	Send message	Connected users	Receivers receive message instantly	Pass
2	Typing Indicator	Sender typing	Typing status shown	Pass
3	User reconnects	Internet restored	Connection re-established automatically	Pass

6.1.4 Integration Testing

Integration testing was conducted after successful completion of unit and functional testing. The objective was to verify that all individual modules interacted correctly when integrated into a complete system. Since KidSafe follows a multi-tier architecture consisting of a frontend, backend, AI microservice, database, SignalR server, and Firebase Cloud Messaging service, integration testing ensured smooth communication among all these components.

The Following Table 6.17 illustrates the Integration test case as shown below

Table 6. 17:Integration Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Login using valid credentials	Registered Child account	User authenticated and redirected to Child Chat page	Pass
2	Send Safe message	Toxicity Score < 0.5	Message delivered successfully, reward points updated, database synchronized	Pass

3	Send Flagged message	Toxicity Score 0.5–0.8	Message masked, stored in database, parent notification generated, dashboard updated	Pass
4	Send Blocked message	Toxicity Score ≥ 0.8	Message blocked, stored in database, notification generated, receiver does not receive message	Pass
5	Parent opens Dashboard	Parent Login	Flagged messages, statistics, and alerts retrieved successfully from database	Pass
6	Teacher opens Teacher Panel	Teacher Login	Classroom statistics and incident reports displayed correctly	Pass
7	Child reaches badge threshold	100 Reward Points	Badge unlocked and Rewards page updated successfully	Pass
8	SignalR real-time messaging	Two connected users	Messages and typing indicators synchronized instantly	Pass
9	Firebase notification delivery	Parent/Teacher subscribed	Push notification successfully delivered on harmful message detection	Pass
10	Complete workflow	Login → Send Message → AI Analysis → Database → Dashboard → Notification	Entire communication pipeline executed successfully without errors	Pass

6.2 Automated Testing

Automated testing was conducted to validate the reliability, performance, and correctness of the KidSafe application. Unlike manual testing, automated testing executes predefined test cases using software tools, reducing human effort and improving testing efficiency. The automated tests focused on validating the REST APIs, AI cyberbullying detection service, authentication mechanism, database connectivity, and overall system performance.

The primary objective of automated testing was to ensure that the application consistently produced accurate results under different operating conditions while maintaining acceptable performance and security standards.

6.2.1 Tools Used

Several testing tools were utilized during the automated testing phase to evaluate different components of the KidSafe application.

The Following Table 6.18 illustrates the Tools Used for Automated Testing as shown below

Table 6. 18:Tools Used for Automated Testing

Tool Name	Tool Description	Applied on (Related Module / FR / NFR)	Results
Swagger UI	Interactive API documentation and testing tool for ASP.NET Core APIs.	Authentication APIs, Chat APIs, Reward APIs, Parent Dashboard APIs	All API endpoints responded successfully with expected HTTP status codes.
FastAPI Swagger Docs	Built-in API testing interface for FastAPI AI service.	AI Message Classification API	Successfully returned toxicity scores and message classifications.
Postman	API testing tool used for sending HTTP requests and validating API responses.	Login API, Registration API, AI Moderation API, Notification API	All APIs returned valid JSON responses with successful execution.
Entity Framework Core	ORM framework used to validate database operations.	CRUD operations on Users, Rewards, and FlaggedMessages tables	Database transactions executed successfully without inconsistencies.
SignalR Testing	Real-time communication testing.	Real-time messaging and typing indicators	Messages were synchronized instantly between connected users.
Firebase Cloud Messaging (FCM)	Push notification testing service.	Parent and Teacher notification module	Notifications were successfully delivered for flagged and blocked messages.

The API testing confirmed that all endpoints returned the expected HTTP response codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, and 404 Not Found) according to different testing scenarios.

The FastAPI AI microservice was tested independently before integration with the backend. Different categories of messages were provided as input to evaluate the performance of the cyberbullying detection model.

The Following Table 6.19 illustrates the AI Microservice Test Results as shown below

Table 6. 19:AI Microservice Test Results

Input Message Type	Expected Classification	Actual Classification	Status
Friendly conversation	Safe	Safe	Pass
Mild offensive language	Flagged	Flagged	Pass
Highly abusive message	Blocked	Blocked	Pass
Neutral statement	Safe	Safe	Pass
Threatening language	Blocked	Blocked	Pass

The AI model successfully identified different categories of cyberbullying content and returned appropriate toxicity scores, confirming the effectiveness of the message moderation module. Authentication testing was performed to verify the correctness of JWT-based security implementation.

The Following Table 6.20 illustrates the Authentication Test Results as shown below

Table 6. 20:Authentication Test Results

Test Scenario	Expected Result	Result
Valid Login	JWT Token Generated	Pass
Invalid Password	Authentication Failed	Pass
Expired JWT Token	Access Denied	Pass
Unauthorized API Access	HTTP 401 Returned	Pass
Authorized API Access	Request Processed Successfully	Pass

The results indicate that JWT authentication successfully protected the application's secure resources against unauthorized access.

Database testing ensured that all records were inserted, updated, retrieved, and deleted correctly without affecting data consistency.

The Following Table 6.21 illustrates the Database Test Results as shown below

Table 6. 21:Database Test Results

Database Operation	Expected Result	Result
Insert User	User stored successfully	Pass
Store Chat Message	Message saved successfully	Pass
Store Flagged Message	Flagged record inserted	Pass
Update Reward Points	Points updated correctly	Pass
Retrieve Dashboard Statistics	Accurate statistics displayed	Pass

All database operations were executed successfully using Entity Framework Core with SQLite. Performance testing was carried out to evaluate the responsiveness of the application under normal operating conditions.

The Following Table 6.22 illustrates the Performance Test Results as shown below

Table 6. 22:Performance Test Results

Performance Metric	Expected Result	Actual Result	Status
Login Response Time	Less than 2 seconds	1.2 seconds	Pass
AI Message Classification	Less than 2 seconds	1.5 seconds	Pass
Chat Message Delivery	Real-time	Instant	Pass
Dashboard Loading	Less than 3 seconds	2.1 seconds	Pass
Notification Delivery	Less than 5 seconds	Approximately 3 seconds	Pass

The application demonstrated satisfactory performance and maintained responsive behavior during testing.

Security testing was performed to verify that sensitive information remained protected throughout the application.

The Following Table 6.23 illustrates the Security Test Results as shown below

Table 6. 23:Security Test Results

Security Test	Expected Result	Result
Password Hashing	Passwords stored securely using BCrypt	Pass
JWT Authentication	Unauthorized users blocked	Pass
Protected API Endpoints	Authentication required	Pass
SQL Injection Attempt	Attack prevented	Pass
Invalid API Request	Proper validation message displayed	Pass

The security testing confirmed that the application successfully implemented secure authentication, password encryption, and request validation mechanisms.

7. Conclusion and Future Work

This chapter presents the conclusion of the KidSafe project and discusses possible enhancements that can be incorporated in future versions of the system. The project successfully achieved its primary objective of providing a secure communication platform for children by utilizing Artificial Intelligence (AI) to detect and prevent cyberbullying in real time. The future work section outlines additional features and technological improvements that can further enhance the functionality, scalability, and effectiveness of the proposed system.

7.1 Conclusion

The increasing use of online communication platforms among children has significantly increased the risk of cyberbullying, exposing young users to harmful language, emotional distress, and psychological issues. Traditional chat applications generally detect inappropriate content only after it has been reported, making them reactive rather than preventive. To address this challenge, this project proposed and implemented KidSafe – An AI-Based Cyberbullying Detection Application, which proactively analyzes chat messages before they reach the intended recipient.

The developed system combines Artificial Intelligence, Natural Language Processing (NLP), real-time communication, and parental monitoring to create a safer digital communication environment for children. The application was developed using Blazor WebAssembly for the frontend, ASP.NET Core Web API for backend services, FastAPI for AI-based message classification, SQLite as the database management system, SignalR for real-time messaging, and Firebase Cloud Messaging (FCM) for instant notifications.

The AI moderation engine successfully classifies every outgoing message into one of three categories: Safe, Flagged, or Blocked. Safe messages are delivered normally while rewarding children with positive points to encourage respectful communication. Flagged messages are partially masked and immediately reported to parents and teachers, whereas highly toxic messages are blocked before reaching the receiver. This proactive moderation mechanism significantly reduces children's exposure to abusive or offensive content.

The developed reward and badge system further motivates children to adopt positive online behavior by recognizing safe communication through points and achievements. Additionally, dedicated dashboards for parents and teachers provide comprehensive monitoring of flagged incidents, toxicity statistics, and notification history, enabling timely intervention whenever inappropriate communication is detected.

Comprehensive manual and automated testing demonstrated that all system modules—including authentication, AI message moderation, real-time messaging, and notification

delivery, reward management, and dashboard functionalities—performed according to the specified functional and non-functional requirements. The successful testing results confirm the reliability, usability, and effectiveness of the proposed solution.

Overall, the KidSafe application successfully fulfills the objectives defined at the beginning of the project by providing an intelligent, secure, and user-friendly platform that protects children from cyberbullying while promoting healthy online interactions. The integration of Artificial Intelligence with modern web technologies demonstrates the practical applicability of AI in addressing real-world social challenges related to children's online safety.

7.2 Future Work

Although the proposed KidSafe application successfully achieves its intended objectives, several enhancements can be incorporated in future versions to improve system performance, scalability, and user experience.

One potential enhancement is the integration of multilingual cyberbullying detection, allowing the AI model to accurately identify harmful messages written in multiple languages such as Urdu, English, Arabic, and regional dialects. This would make the application more suitable for deployment in multilingual environments.

The accuracy of cyberbullying detection can also be improved by training the AI model on larger and more diverse datasets containing real-world cyberbullying conversations. Continuous model retraining would enable the system to recognize newly emerging abusive language and slang expressions more effectively.

Future versions of the system may incorporate voice message moderation, where speech-to-text technology converts voice messages into text before they are analyzed by the AI model. Similarly, image and multimedia content moderation can be implemented using computer vision techniques to detect offensive images, memes, or inappropriate visual content shared within conversations.

The reward mechanism may also be expanded by introducing personalized learning activities, behavioral analytics, achievement levels, and interactive educational games that promote responsible digital citizenship among children.

Another important enhancement would be the development of dedicated Android and iOS mobile applications, enabling users to access KidSafe through smartphones with improved performance, offline support, and enhanced notification capabilities.

The parent and teacher dashboards can be extended with advanced analytical features, including predictive risk analysis, behavioral trend visualization, weekly safety reports, and AI-generated recommendations for early intervention.

Cloud-based deployment using platforms such as Microsoft Azure, Amazon Web Services (AWS), or Google Cloud Platform (GCP) can further improve system

scalability, availability, and reliability, allowing the application to support a larger number of concurrent users.

Future implementations may also integrate large language models (LLMs) to better understand conversational context, sarcasm, implicit threats, and contextual cyberbullying that traditional text classification models may fail to detect. This would significantly enhance the intelligence and accuracy of the moderation process.

Finally, the application can be integrated with educational institutions and child safety organizations to provide centralized monitoring, awareness programs, and policy-based reporting mechanisms that contribute to creating a safer digital ecosystem for children.

In conclusion, the proposed KidSafe application establishes a strong foundation for AI-driven cyberbullying prevention. By incorporating the suggested enhancements, future versions can evolve into a more intelligent, scalable, and comprehensive child online safety platform capable of addressing the continuously changing landscape of cyber threats.

8. References

- [1] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [2] T. Wolf, L. Debut, V. Sanh, et al., "Transformers: State-of-the-Art Natural Language Processing," *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020, pp. 38–45.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [5] I. Sommerville, *Software Engineering*, 10th ed. Harlow, England: Pearson Education, 2016.
- [6] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.
- [7] Microsoft Corporation, "ASP.NET Core Documentation." Available: <https://learn.microsoft.com/aspnet/core>
- [8] Microsoft Corporation, "Blazor WebAssembly Documentation." Available: <https://learn.microsoft.com/aspnet/core/blazor>
- [9] Microsoft Corporation, "ASP.NET Core SignalR Documentation." Available: <https://learn.microsoft.com/aspnet/core/signalr>
- [10] Microsoft Corporation, "Entity Framework Core Documentation." Available: <https://learn.microsoft.com/ef/core>
- [11] FastAPI, "FastAPI Documentation." Available: <https://fastapi.tiangolo.com>
- [12] Hugging Face, "Transformers Documentation." Available: <https://huggingface.co/docs/transformers>
- [13] Hugging Face, "Hugging Face Hub Documentation." Available: <https://huggingface.co/docs/hub>
- [14] Google LLC, "Firebase Cloud Messaging Documentation." Available: <https://firebase.google.com/docs/cloud-messaging>
- [15] SQLite Development Team, "SQLite Documentation." Available: <https://www.sqlite.org/docs.html>

- [16] JWT.IO, "JSON Web Token (JWT) Introduction." Available: <https://jwt.io/introduction>
- [17] OWASP Foundation, "OWASP Top 10: Web Application Security Risks." Available: <https://owasp.org/www-project-top-ten/>
- [18] Python Software Foundation, "Python 3 Documentation." Available: <https://docs.python.org/3>
- [19] Microsoft Corporation, "Blazor WebAssembly Security Documentation." Available: <https://learn.microsoft.com/aspnet/core/blazor/security>
- [20] Microsoft Corporation, "ASP.NET Core Authentication and Authorization." Available: <https://learn.microsoft.com/aspnet/core/security/authentication>